ENTRADA-SALIDA (I/O)

Además del procesador y los módulos de memoria, otro componente esencial de una computadora es el conjunto de módulos de entrada/salida. Cada uno de estos módulos se conecta al bus del sistema o a un conmutador central y gestiona uno o varios dispositivos periféricos. Un módulo de entrada/salida no es simplemente un conector mecánico que permite enchufar el dispositivo al bus del sistema; también incorpora una cierta "inteligencia". Esto significa que contiene la lógica necesaria para facilitar la comunicación eficiente entre el periférico y el bus.

Módulos de entrada/salida:

Las principales funciones y requisitos de un módulo de entrada/salida se agrupan en las siguientes categorías:

- Control y temporización
- Comunicación con el procesador
- Comunicación con los dispositivos
- Almacenamiento temporal de datos
- Detección de errores

En cualquier momento, el procesador puede comunicarse con uno o más dispositivos externos en cualquier orden, según las necesidades de entrada/salida del programa. Los recursos internos, como la memoria principal y el bus del sistema, deben compartirse entre distintas actividades, incluida la entrada/salida de datos. Por lo tanto, la función de entrada/salida incluye ciertos requisitos de control y temporización para coordinar el tráfico entre los recursos internos y los dispositivos externos.

Escritura de un módulo de entrada/salida:

El módulo de entrada/salida se conecta al resto de la computadora a través de un conjunto de líneas de bus. Los datos que se transfieren hacia y desde el módulo se almacenan **temporalmente en uno o más registros de datos**. Además, puede haber uno o más **registros de estado** que proporcionan información sobre el estado actual. **Un registro de estado también puede funcionar como un registro de control,** recibiendo información de control del procesador. La lógica en un módulo interactúa con el procesador **a través de una serie de líneas de control.** Estas son utilizadas por el procesador para proporcionar **órdenes** al módulo de entrada/salida. Algunas de las líneas de control pueden ser utilizadas por el propio módulo de entrada/salida.
El módulo también debe ser capaz de reconocer y generar las **direcciones** asociadas a los dispositivos que controla. Cada módulo de entrada/salida tiene **una dirección única o, si controla más de un dispositivo externo, un conjunto único de direcciones**. Por último, el módulo de entrada/salida posee la lógica específica para la interfaz con cada uno de los dispositivos que controla.

Técnicas de entrada/salida: Existen tres técnicas posibles para las operaciones de entrada/salida:

- **Entrada/salida programada:** Los datos se intercambian entre el procesador y el módulo de entrada/salida. El procesador ejecuta un programa que controla directamente la operación de entrada/salida, incluyendo la comprobación del estado del dispositivo, el envío de una orden de lectura o escritura y la transferencia de datos. Cuando el procesador envía una orden al módulo de entrada/salida, debe esperar hasta que la operación de entrada/salida se complete.

- **Entrada/salida mediante interrupciones:** El procesador proporciona la orden de entrada/salida y continúa ejecutando otras instrucciones. El módulo de entrada/salida interrumpe al procesador cuando ha terminado su trabajo. Tanto en la entrada/salida programada como en la entrada/salida mediante interrupciones, el procesador es responsable de extraer los datos de la memoria principal para una operación de salida y de almacenar los datos en la memoria principal para una operación de entrada.

- **DMA (Acceso Directo a Memoria)**: Es una alternativa en la que el módulo de entrada/salida y la memoria principal intercambian datos directamente, sin la intervención del procesador. En esta técnica, el procesador

solo inicia la transferencia y es liberado para realizar otras tareas mientras la operación de entrada/salida se lleva a cabo de manera autónoma por el módulo de DMA.

Técnicas de entrada/salida (más a detalle):

E/S programadas:

Cuando el procesador está ejecutando un programa y encuentra una instrucción relacionada con una entrada/salida, ejecuta dicha instrucción enviando una orden al módulo de entrada/salida apropiado. Con entrada/salida programada, el módulo de entrada/salida realizará la acción solicitada y después activará los bits apropiados en el registro de estado de entrada/salida. El módulo de entrada/salida no realiza ninguna otra acción para avisar al procesador. En concreto, no interrumpe al procesador. De esta forma, el procesador es responsable de comprobar periódicamente el estado del módulo de entrada/salida hasta que encuentre que la operación ha terminado.

Al ejecutar una instrucción relacionada con una entrada/salida, el procesador proporciona una dirección especificando el módulo de entrada/salida particular y el dispositivo externo, así como una orden de entrada/salida. Hay 4 tipos de órdenes:

- **Control**: Se utiliza para activar el periférico e indicarle qué hacer.
- **Test**: Se utiliza para comprobar diversas condiciones de estado asociadas con el módulo de entrada/salida y sus periféricos.
- **Lectura**: Hace que el módulo capte un dato de un periférico y lo sitúe en el buffer interno.
- **Escritura**: Hace que el módulo de entrada/salida capte un dato del bus de datos y lo transmita al periférico.

Normalmente, habrá muchos dispositivos de entrada/salida conectados al sistema a través de los módulos de entrada/salida. Cada dispositivo tiene asociado un identificador único o dirección. Cuando el procesador envía una orden de entrada/salida, la orden contiene la dirección del dispositivo deseado. Así, cada módulo de entrada/salida debe interpretar las líneas de dirección para determinar si la orden es para él.

Cuando el procesador, la memoria principal y las entradas/salidas comparten un bus común, son posibles dos modos de direccionamiento:

- **Asignado en memoria:** Existe un único espacio de direcciones para las posiciones de memoria y los dispositivos de entrada/salida. El procesador considera los registros de estado y de datos de los módulos de entrada/salida como posiciones de memoria y utiliza las mismas instrucciones máquina para acceder tanto a la memoria como a los dispositivos de entrada/salida.
- **Aislado**: Con las entradas/salidas asignadas en memoria se necesitará una sola línea de lectura y una sola de escritura en el bus. Alternativamente, el bus puede disponer de líneas de lectura y escritura en memoria junto con líneas para órdenes de entrada y salida. En este caso, las líneas de órdenes especifican si las direcciones se refieren a una posición de memoria o a un dispositivo de entrada/salida. Como el espacio de direcciones de entrada/salida está aislado del de memoria, este modo de direccionamiento se conoce como 'entrada/salida aislada'.

E/S con interrupciones:

El problema de la entrada/salida programada es que el procesador tiene que esperar un tiempo considerable a que el módulo de entrada/salida en cuestión esté preparado para recibir o transmitir los datos. Mientras espera, el procesador debe comprobar repetidamente el estado del módulo de entrada/salida. Como consecuencia, se degrada el nivel de prestaciones de todo el sistema.

Una alternativa consiste en que el procesador, tras enviar una orden de entrada/salida a un módulo, continúe realizando algún trabajo útil. Después, el módulo de entrada/salida interrumpirá al procesador para solicitar su servicio cuando esté preparado para intercambiar datos con él. El procesador entonces ejecuta la transferencia de datos, como antes, y después continúa con el procesamiento previo.

Supongamos que un módulo de entrada/salida recibe una orden de lectura del procesador. Entonces, el módulo de entrada/salida procede a leer el dato desde el periférico asociado. Una vez que el dato está en el registro de datos del módulo, éste envía una interrupción al procesador a través de una línea de control. Luego, espera a que el procesador

solicite su dato. Cuando ha recibido la solicitud, el módulo sitúa su dato en el bus de datos y pasa a estar preparado para otra operación de entrada/salida.
Desde el punto de vista del procesador, las acciones para una entrada son: el procesador envía una orden de lectura y pasa a hacer otro trabajo. Al final de cada ciclo de instrucciones, el procesador comprueba las interrupciones. Cuando se pide la interrupción desde el módulo de entrada/salida, el procesador guarda el contenido (el PC y los demás registros) del programa actual y procesa la interrupción (lee la palabra de datos del módulo de entrada/salida y la almacena en memoria). Luego recupera el contexto y continúa con su ejecución.
En la implementación de las entradas/salidas mediante interrupciones aparecen dos cuestiones. Primero, como que casi invariablemente habrá múltiples módulos de entrada/salida, ¿cómo determina el procesador qué dispositivo ha provocado la interrupción? Y segundo, si se producen varias, ¿cómo decide el procesador las que debe atender? Consideremos en primer lugar la identificación del dispositivo. **Se utilizan 4 técnicas comúnmente**:

- **Múltiples líneas de interrupción:** poco práctico por la posible gran cantidad de módulos.
- **Consulta software (polling)**: al detectar una interrupción, se produce una bifurcación a una rutina de servicio de interrupción que se encarga de consultar a cada módulo de entrada/salida para determinar cuál la eligió.
- **Conexión en cadena**: como la consulta software consume mucho tiempo, surge la conexión en cadena de los módulos de entrada/salida que nos proporciona una consulta hardware. Entonces, cuando el procesador recibe una interrupción, activa el reconocimiento de interrupción. Esta señal se propaga a través de la secuencia de módulos hasta que llega al que solicitó la interrupción. Este módulo responde colocando una palabra en las líneas de datos, denominada "vector", y que es la dirección del módulo de entrada/salida o algún otro tipo de identificador específico. El procesador utiliza el vector como un puntero a la rutina de servicio de dispositivo apropiada. Esta técnica se conoce como "interrupción vectorizada".
- **Arbitraje de bus**: con el arbitraje de bus, un módulo de entrada/salida debe en primer lugar disponer del control del bus antes de poder activar la línea de petición de interrupción. Así, sólo un módulo puede activar la línea en un instante. Cuando el procesador detecta la interrupción, responde mediante la línea de reconocimiento de interrupción. Después, el módulo que solicitó la interrupción sitúa su vector en las líneas de datos.

Acceso Directo a Memoria (DMA):

La entrada/salida con interrupciones, aunque más eficiente que la programada, también requiere la intervención activa del procesador para transferir datos entre la memoria y el módulo de entrada/salida, y cualquier transferencia de datos debe seguir un camino a través del procesador.
Cuando hay que transferir grandes volúmenes de datos, se requiere una técnica más eficiente: el acceso directo a memoria (DMA). El DMA requiere un módulo adicional en el bus del sistema. Dicho módulo es capaz de imitar al procesador y, de hecho, de recibir el control del sistema cedido por el procesador (necesario para transferir datos a, y desde, la memoria a través del bus del sistema). Para hacerlo, el módulo de DMA debe utilizarse sólo cuando el procesador no lo necesita, o debe forzar al procesador a que suspenda temporalmente su funcionamiento. Esta última técnica es la más común y se denomina "robo de ciclo" puesto que, en efecto, el módulo de DMA roba un ciclo del bus.
Cuando el procesador desea leer o escribir un bloque de datos, envía una orden al módulo de DMA, incluyendo la siguiente información:

- **Si se solicita una lectura o escritura**, utilizando la línea de control correspondiente.
- **Dirección del dispositivo** de entrada/salida, indicada a través de líneas de datos.
- **Posición inicial de memoria** a partir de la que se lee o escribe, indicada a través de las líneas de datos y almacenada por el módulo de DMA en su registro de direcciones.
- **Número de palabras** a leer o escribir, almacenado en el registro de cuenta de datos e indicado a través de las líneas de datos.

Después, el procesador continúa con otro trabajo. Ha delegado la operación de entrada/salida al módulo de DMA, que se encargará de ella. El módulo de DMA transfiere el bloque completo de datos, palabra a palabra, directo desde o hacia la memoria, sin que tenga que pasar a través del procesador. Cuando la transferencia se ha terminado, el módulo

de DMA envía una señal de interrupción al procesador. De esta forma, el procesador solo interviene al comienzo y al final de la transferencia.
No se trata de una interrupción de la acción en sí; el procesador no guarda el contexto ni hace nada más cuando el módulo de DMA le roba el ciclo. En cambio, espera durante un ciclo de bus. El efecto resultante es que el procesador es más lento ejecutando programas. El número de ciclos de bus necesarios puede reducirse sustancialmente si se integran las funciones de DMA y entrada/salida.
Esto significa que existe un camino entre el módulo de DMA y uno o más módulos de Entrada/salida que no incluye al bus del sistema. La lógica de DMA puede ser parte de un módulo de entrada/salida, o puede ser un módulo separado que controla a uno o más módulos de entrada/salida. Este concepto se puede llevar algo más lejos conectando los módulos de entrada/salida al DMA mediante un bus de entrada/salida en el módulo de entrada/salida y produce una configuración fácilmente ampliable.
En todos estos casos el bus del sistema, que el módulo de DMA comparte con el procesador y la memoria, es usado por el módulo de DMA solo para intercambiar datos con la memoria. El intercambio de datos entre los módulos de DMA y entrada/salida se produce fuera del bus del sistema.

INTERRUPCIONES

Las interrupciones son cambios en el flujo de control causados no por el programa en ejecución sino por otra cosa, casi siempre relacionado con E/S. Por ejemplo, un programa podría ordenar al disco que comience a transferir información y que genere una interrupción apenas haya terminado la transferencia. La interrupción detiene el programa en ejecución y transfiere el control a un manejador de interrupciones, el cual realiza alguna acción apropiada. Cuando el manejador de interrupción termina, devuelve el control al programa interrumpido. El proceso interrumpido se debe reiniciar en el mismo estado exactamente en el que estaba cuando ocurrió la interrupción, lo que implica restaurar todos los registros internos a su estado previo a la interrupción. Las interrupciones son **asincrónicas**, o sea, ocurren en cualquier momento, ya que no son producidas por el programa en ejecución.

Mecanismo de las interrupciones:

1. El controlador de dispositivo habilita una línea de interrupción en el bus del sistema para iniciar la secuencia de interrupción.
2. Tan pronto como la CPU está lista para manejar la interrupción, habilita una señal de reconocimiento de interrupción en el bus.
3. Cuando el controlador de dispositivo ve que su señal de interrupción ha sido reconocida, coloca un entero pequeño en las líneas de datos para identificarse. Este número se llama vector de interrupción.
4. La CPU quita el vector de interrupción del bus y lo guarda temporalmente.
5. Luego, la CPU almacena el contador de programa y la palabra de estado del programa en la pila.
6. La CPU localiza un nuevo contador de programa utilizando el vector de interrupción como índice de una tabla que está en la parte baja de la memoria.
7. Lo primero que hace la rutina de servicio de interrupción es guardar todos los registros para que puedan restaurarse después. Se pueden guardar en la pila o en una tabla del sistema.
8. En general, cada vector de interrupción es compartido por todos los dispositivos de un tipo de dato, por lo que todavía no se sabe cuál terminal causó la interrupción. El número de terminal puede averiguarse leyendo algún registro del dispositivo.
9. Ahora puede leerse cualquier otra información que haya acerca de la interrupción, como códigos de situación.
10. Si ocurrió un error de E/S, se puede manejar aquí.
11. Se actualizan las variables globales de apuntadores y contadores.
12. Si es necesario, se envía a la salida un código especial para indicar al dispositivo que ya se procesó la interrupción.
13. Se restauran todos los registros que se guardaron.

14. Se ejecuta una instrucción especial, que coloca a la CPU otra vez en el modo y el estado en que estaba antes de la interrupción. La computadora continúa con lo que estaba haciendo.

Un concepto fundamental relacionado con las interrupciones es la **transparencia**. Cuando ocurre una interrupción, se efectúan algunas acciones y se ejecuta cierto código, pero una vez que ha terminado, la computadora deberá volver al mismo estado exactamente en el que estaba antes de la interrupción.

Las interrupciones anidadas:

Las interrupciones anidadas permiten que una interrupción sea interrumpida por otra de mayor prioridad, creando una pila de interrupciones donde cada nivel guarda su propio estado. Este mecanismo asegura que eventos más críticos puedan ser atendidos de inmediato. Cuando una interrupción de mayor prioridad ocurre durante el manejo de otra interrupción, el estado actual de la CPU, incluyendo registros, contador de programa y palabra de estado del programa, se guarda en una estructura de pila. Esta pila sigue el principio LIFO (Last In, First Out), lo que significa que la última interrupción en ser atendida será la primera en ser completada.
Por ejemplo, si una interrupción de baja prioridad está en curso y ocurre una interrupción de alta prioridad, el sistema guardará el estado de la interrupción en curso y atenderá primero la interrupción de mayor prioridad. Una vez que esta interrupción de alta prioridad se ha manejado, el sistema restaurará el estado previo desde la pila y continuará con la interrupción original. Este proceso permite que las interrupciones sean manejadas de manera ordenada y eficiente, asegurando que cada interrupción sea atendida en el orden correcto y que el sistema vuelva al estado apropiado una vez que todas las interrupciones han sido manejadas.

Las interrupciones secuenciales:

Las interrupciones secuenciales se manejan una a la vez, completando la ISR (Interrupt Service Routine) de una antes de atender la siguiente. Aunque este método es más simple, puede no ser tan eficiente en sistemas con muchas interrupciones de diferentes prioridades. En estos casos, el sistema debe esperar a que se complete la interrupción actual antes de atender la siguiente, lo que puede resultar en demoras significativas si una interrupción de alta prioridad ocurre mientras se maneja una de baja prioridad. Esto puede afectar negativamente la capacidad de respuesta del sistema y su desempeño general, especialmente en aplicaciones de tiempo real donde la rapidez es crítica.

Ampliación de Conceptos: Prioridades, Rutinas de Servicio (ISR), su ubicación y acceso a ellas:

Prioridades: En un sistema de interrupciones, es crucial tener un mecanismo para priorizar y manejar las interrupciones más urgentes de manera eficiente. Existen varias alternativas para la gestión de interrupciones:

- **Interrupciones Anidadas:** Permiten que una interrupción en curso sea interrumpida por otra de mayor prioridad. Esto se logra guardando el estado de la interrupción actual y atendiendo primero la interrupción más crítica. Este método asegura que las interrupciones más urgentes sean tratadas inmediatamente, mejorando la capacidad de respuesta del sistema.
- **Inhabilitación de Interrupciones:** Consiste en deshabilitar todas las interrupciones mientras se está atendiendo una. Aunque este enfoque simplifica el manejo de interrupciones, puede resultar ineficiente en sistemas donde las interrupciones de alta prioridad deben ser atendidas sin demora.

Tener interrupciones manejadas por prioridad es más efectivo ya que permite atender interrupciones críticas sin tener que esperar a que se completen las de menor prioridad. Esto logra un equilibrio en la atención de diferentes interrupciones y mejora el rendimiento del sistema en aplicaciones de tiempo real.

Rutinas de Servicio de Interrupciones (ISR): Las Rutinas de Servicio de Interrupciones (ISR) son programas especiales diseñados para manejar interrupciones específicas. Cada ISR tiene una función definida para gestionar el evento que causó la interrupción y restaurar el sistema a su estado normal de operación.

- **Ubicación de las ISR:** Las direcciones de las ISR están almacenadas en una estructura conocida como vector de interrupciones, que es una tabla en memoria. Cada entrada en esta tabla contiene la dirección de inicio de una ISR específica.
- **Acceso a las ISR:** Cuando ocurre una interrupción, el procesador consulta el vector de interrupciones utilizando el número de interrupción para obtener la dirección de la ISR correspondiente. Este número de interrupción actúa como un índice en la tabla del vector. El procesador luego transfiere el control a la ISR para manejar la interrupción.

El diseño eficiente y la correcta ubicación de las ISR son cruciales para un manejo efectivo de las interrupciones. Al tener una tabla centralizada (vector de interrupciones), el sistema puede acceder rápidamente a las ISR necesarias, garantizando una respuesta rápida y adecuada a los eventos de interrupción.

Beneficios de la Prioridad en Interrupciones y ISR:

- **Mejora de la Respuesta del Sistema:** Las interrupciones más urgentes se manejan de inmediato, mejorando la responsividad del sistema.
- **Optimización de Recursos:** Al permitir interrupciones anidadas, los recursos del sistema se utilizan de manera más eficiente.
- **Mantenimiento de la Integridad del Sistema:** La estructura del vector de interrupciones y el manejo adecuado de las ISR aseguran que el sistema vuelva a su estado normal después de manejar las interrupciones.
- **Flexibilidad en el Manejo de Interrupciones:** Permite una jerarquía flexible y eficiente para manejar múltiples interrupciones con diferentes niveles de prioridad.

Técnicas para Determinar el Dispositivo que Provocó la Interrupción:

- **Múltiples Líneas de Interrupción:** Se proporcionan varias líneas de interrupción entre el procesador y los módulos de E/S. No es práctica debido a la posible conexión de múltiples módulos de E/S.
- **Consulta Software (polling):** La ISR consulta a cada módulo de E/S para determinar quién provocó la interrupción. Cada módulo de E/S podría tener un registro de estado para identificación.
- **Consulta Hardware:** Los módulos de E/S comparten una línea común para solicitar interrupciones y están encadenados. El procesador recibe la interrupción y activa el reconocimiento de interrupción. La señal de reconocimiento se propaga hasta encontrar el módulo que solicitó la interrupción, el cual responde con un **vector** que el procesador usa como puntero a la ISR.
- **Arbitraje de Bus:** El módulo debe disponer del control del bus antes de activar la línea de petición de interrupción. Solo un módulo puede activar la línea en un instante. El procesador detecta la interrupción y responde mediante la línea de reconocimiento de interrupción.

SECUENCIADORES

Un secuenciador es un **circuito digital que controla el orden en que se ejecutan las instrucciones en un microcontrolador o microprocesador**. Es responsable de **generar las señales de control necesarias para dirigir el flujo de datos y operaciones dentro del procesador, asegurando que cada instrucción se ejecute en el momento adecuado y en el orden correcto**.

Gobiernan todos los componentes de la CPU, y emite las microordenes (señales) a cada uno de ellos que sincronizadas con los pulsos de reloj hace que se vayan ejecutando paso a paso las instrucciones cargadas en el registro de instrucciones (RI).

Secuenciadores de tipo cableados:

Los secuenciadores de tipo cableado están **constituidos principalmente** por **puertas lógicas y otros componentes de lógica combinacional**, formando **circuitos secuenciales electrónicos** que **generan las *señales de control*** necesarias

para ejecutar instrucciones en microcontroladores o microprocesadores. Su control se basa en una **arquitectura fija**, lo que requiere cambios en el cableado si cambia el conjunto de instrucciones. Son utilizados principalmente en computadoras **RISC**, que consisten en un conjunto pequeño de instrucciones.
Estos secuenciadores se caracterizan por su diseño rígido pero **extremadamente rápido**. La velocidad de ejecución es una de sus principales ventajas, ya que **las señales de control se generan directamente a través de circuitos dedicados, sin necesidad de decodificación adicional**. Esto los hace ideales para aplicaciones donde el rendimiento es crítico y el conjunto de instrucciones es relativamente estable.
La naturaleza fija de su arquitectura implica que **cada instrucción tiene un camino de ejecución predefinido y optimizado en hardware**. Esto permite una ejecución muy eficiente de las instrucciones soportadas, lo que es particularmente beneficioso en arquitecturas RISC donde se busca maximizar la velocidad de procesamiento.
Sin embargo, la rigidez de su diseño también presenta desafíos. **Modificar el conjunto de instrucciones en un secuenciador cableado es un proceso complejo que implica cambios físicos en el hardware**. Esto limita la flexibilidad y adaptabilidad del sistema, haciendo que sea menos adecuado para entornos donde se requieren actualizaciones frecuentes del conjunto de instrucciones.
En el contexto de las arquitecturas RISC, los secuenciadores cableados encuentran su aplicación ideal. Estas arquitecturas, con su enfoque en un conjunto reducido de instrucciones simples y altamente optimizadas, se benefician enormemente de la velocidad y eficiencia que ofrecen los secuenciadores cableados. La simplicidad relativa del conjunto de instrucciones RISC se alinea bien con la **naturaleza fija** de estos secuenciadores, permitiendo una implementación eficiente y de alto rendimiento.
A pesar de sus limitaciones en términos de flexibilidad, los secuenciadores cableados siguen siendo una opción atractiva en sistemas donde la velocidad de ejecución y la eficiencia energética son prioritarias sobre la capacidad de modificación del conjunto de instrucciones. Su uso continúa siendo relevante en aplicaciones específicas y en sistemas embebidos donde el conjunto de tareas está bien definido y es poco probable que cambie significativamente durante la vida útil del dispositivo.

Secuenciadores microprogramados:

Los secuenciadores microprogramados son una **alternativa más flexible que utiliza una memoria ROM con microinstrucciones**, donde **cada instrucción se ejecuta por microprogramas que generan las señales correspondientes para los componentes de la CPU**, permitiendo modificar el microprograma para cambiar el conjunto de instrucciones, aunque este diseño es más lento y desapareció con la arquitectura de las máquinas RISC.
El funcionamiento de los secuenciadores microprogramados **se basa en la programación de transferencias entre registros y la secuencia en que estas se deben efectuar**. Utilizan una memoria adicional para alojar el microprograma, que contiene microinstrucciones para cada instrucción. Cada microinstrucción genera una o varias microórdenes.

$$Memoria\ ROM\ (Aloja\ al) \rightarrow Microprograma\ (Contiene) \rightarrow Microinstrucción\ (Genera) \rightarrow Microorden(Señal\ de\ control)$$

Esta arquitectura ofrece una gran flexibilidad en el diseño y la implementación de conjuntos de instrucciones. Al almacenar las microinstrucciones en una memoria separada, **es posible modificar el comportamiento de las instrucciones existentes o incluso añadir nuevas instrucciones sin cambiar el hardware subyacente**. Esto permite una adaptabilidad significativa a diferentes requisitos de programación y facilita la creación de instrucciones complejas y poderosas.
La flexibilidad de los secuenciadores microprogramados los hace particularmente útiles en **entornos donde el conjunto de instrucciones puede necesitar actualizaciones o modificaciones frecuentes.** También **son ventajosos en situaciones donde se requieren instrucciones altamente especializadas o complejas que serían difíciles de implementar** eficientemente en hardware fijo.
Sin embargo, **esta flexibilidad viene con un costo en términos de velocidad de ejecución**. Los secuenciadores microprogramados son inherentemente más lentos que sus contrapartes cableadas. Esto se debe a que cada instrucción **requiere múltiples accesos a la memoria de microprograma para recuperar y ejecutar las microinstrucciones** correspondientes, lo que **introduce latencias adicionales** en el proceso de ejecución.

La llegada de las arquitecturas RISC (Reduced Instruction Set Computing) marcó un punto de inflexión para los secuenciadores microprogramados. Las arquitecturas RISC, con su enfoque en un conjunto reducido de instrucciones simples que se pueden ejecutar rápidamente en hardware, redujeron significativamente la necesidad de la flexibilidad ofrecida por la microprogramación. La simplicidad y la velocidad de ejecución de las instrucciones RISC superaron las ventajas de la flexibilidad de los secuenciadores microprogramados en muchas aplicaciones.
Aunque los secuenciadores microprogramados han sido en gran medida reemplazados en los procesadores de propósito general modernos, su legado persiste en ciertos nichos. Algunos sistemas especializados, particularmente aquellos que requieren conjuntos de instrucciones altamente personalizados o que necesitan mantener compatibilidad con sistemas legacy, aún pueden encontrar útil la flexibilidad ofrecida por la microprogramación.

Característica	Secuenciadores Cableados	Secuenciadores Microprogramados
Componentes Principales	Puertas lógicas y componentes de lógica combinacional.	Memoria ROM que almacena microinstrucciones.
Control	Basado en una arquitectura fija, requiere cambios en el cableado para modificar el conjunto de instrucciones.	Basado en microprogramas almacenados en memoria ROM, permite modificar el conjunto de instrucciones sin cambiar el hardware.
Paralelismo	No explota el paralelismo, cada instrucción tiene un camino de ejecución predefinido en hardware.	Cada instrucción se ejecuta por microprogramas que generan señales para los componentes de la CPU.
Velocidad de Ejecución	Muy rápida, las señales de control se generan directamente a través de circuitos dedicados sin necesidad de decodificación adicional.	Más lenta, requiere múltiples accesos a la memoria de microprograma para ejecutar microinstrucciones, introduciendo latencias adicionales.
Flexibilidad	Rígido, modificar el conjunto de instrucciones es complejo y requiere cambios físicos en el hardware.	Alta flexibilidad, permite modificar el microprograma para cambiar el conjunto de instrucciones sin alterar el hardware subyacente.
Uso Típico	Principalmente en computadoras RISC con un conjunto pequeño y estable de instrucciones.	Útil en entornos que requieren actualizaciones frecuentes del conjunto de instrucciones o instrucciones complejas y especializadas.
Ventajas	- Diseño sencillo. - Ejecución extremadamente rápida.	- Gran flexibilidad. - Adaptabilidad para actualizar o añadir nuevas instrucciones.
Limitaciones	- Difícil de modificar. - Menos adecuado para sistemas que requieren actualizaciones frecuentes.	- Menor velocidad de ejecución comparada con secuenciadores cableados. - Más complejo en términos de diseño y gestión de microprogramas.
Impacto de las Arquitecturas RISC	Ideales para arquitecturas RISC que buscan maximizar la velocidad de procesamiento con un conjunto reducido de instrucciones.	La llegada de las arquitecturas RISC redujo su uso, ya que la simplicidad y velocidad de las instrucciones RISC disminuyeron la necesidad de la flexibilidad de la microprogramación.

Modelo de Wilkes:

Es una técnica crucial en el diseño de **secuenciadores microprogramados**, introducida por Maurice Wilkes, un pionero en la computación. Este modelo, también conocido como **secuenciamiento explícito**, **busca mejorar el diseño de la unidad de control convirtiéndola en una computadora en miniatura que utiliza dos tablas: una para especificar el control de datos y otra para determinar el flujo de control a nivel de microinstrucción.** Este enfoque está estrechamente relacionado con la **codificación horizontal**, en la que cada microinstrucción contiene bits que controlan directamente los diferentes componentes del procesador y la dirección de la siguiente microinstrucción, proporcionando así un control explícito y detallado sobre el flujo de ejecución.

Direccionamiento y secuenciamiento:

Implícito: En este método, **la siguiente microinstrucción a ejecutar se encuentra físicamente después de la actual en la memoria de control**. No contiene la dirección de la siguiente microinstrucción, lo que **requiere un contador de microprograma**. Es más lento debido a **las bifurcaciones que consumen ciclos adicionales de reloj**.

El direccionamiento implícito **se basa en la idea de secuencialidad natural de las instrucciones**. Funciona de manera similar a cómo un procesador convencional **ejecuta instrucciones** de máquina en **secuencia**. *El contador de microprograma se incrementa automáticamente después de cada microinstrucción, apuntando a la siguiente en memoria*.

Este enfoque tiene la ventaja de **simplificar el diseño de las microinstrucciones**, ya que no necesitan incluir información sobre la siguiente dirección. Esto resulta en **microinstrucciones más cortas y una memoria de control más compacta.** Sin embargo, la principal desventaja surge cuando se necesitan realizar **saltos** o **bifurcaciones en el flujo de ejecución.**

Para manejar bifurcaciones, el direccionamiento implícito generalmente **requiere microinstrucciones adicionales dedicadas específicamente a realizar el salto**. Estas microinstrucciones de salto **consumen ciclos de reloj adicionales**, lo que puede **ralentizar** la ejecución del microprograma, especialmente en situaciones donde las bifurcaciones son frecuentes.

Explícito: En este método, **cada microinstrucción indica la dirección de la siguiente microinstrucción**, lo que **aumenta** el **tamaño de la microinstrucción y la memoria de control**. Este método es más **eficiente** para manejar bifurcaciones, **evitando ciclos adicionales** de **reloj**.

El direccionamiento explícito, también conocido como el **Modelo de Wilkes**, proporciona un **control directo sobre el flujo de ejecución**. Cada microinstrucción contiene un campo que especifica la dirección de la siguiente microinstrucción a ejecutar. Esto elimina la necesidad de un contador de microprograma que se incremente secuencialmente.

La principal ventaja de este enfoque es su **eficiencia en el manejo de bifurcaciones y saltos**. Como la dirección de la siguiente microinstrucción está siempre disponible, **no se requieren ciclos adicionales para calcular o cargar la dirección del salto**. Esto resulta en una ejecución más rápida de secuencias de microinstrucciones que involucran muchas bifurcaciones.

Sin embargo, el direccionamiento explícito tiene sus desventajas. **El tamaño de cada microinstrucción aumenta significativamente debido a la inclusión de la dirección de la siguiente instrucción**. Esto resulta en un mayor uso de memoria para el microprograma completo. Además, el diseño del hardware para manejar este tipo de direccionamiento puede ser **más complejo**.

Característica	Direccionamiento Implícito	Direccionamiento Explícito
Definición	La siguiente microinstrucción se encuentra físicamente después de la actual en la memoria de control.	Cada microinstrucción indica explícitamente la dirección de la siguiente microinstrucción.
Contador de Microprograma	Requiere un contador de microprograma que se incrementa automáticamente después de cada microinstrucción.	No requiere contador de microprograma, la dirección está incluida en cada microinstrucción.
Velocidad de Ejecución	Más lenta debido a las bifurcaciones que consumen ciclos adicionales de reloj.	Más rápida en el manejo de bifurcaciones, evita ciclos adicionales de reloj.
Simplicidad del Diseño	Simplifica el diseño de las microinstrucciones y resulta en una memoria de control más compacta.	El diseño es más complejo debido a la inclusión de la dirección de la siguiente microinstrucción en cada instrucción.
Tamaño de las Microinstrucciones	Las microinstrucciones son más cortas porque no contienen la dirección de la siguiente instrucción.	Las microinstrucciones son más largas debido a la inclusión de la dirección de la siguiente instrucción.
Eficiencia en Bifurcaciones	Requiere microinstrucciones adicionales para manejar bifurcaciones, lo que puede ralentizar la ejecución.	Maneja eficientemente las bifurcaciones y saltos sin necesidad de ciclos adicionales.

Uso de Memoria	Utiliza menos memoria debido a la ausencia de direcciones explícitas en las microinstrucciones.	Utiliza más memoria debido a las direcciones explícitas en cada microinstrucción.
Complejidad del Hardware	Menos complejo, ya que no necesita manejar direcciones explícitas.	Más complejo, debido a la necesidad de manejar direcciones explícitas en cada microinstrucción.
Aplicación Típica	Adecuado para secuencias lineales de microinstrucciones con pocas bifurcaciones.	Adecuado para secuencias con muchas bifurcaciones y saltos, donde la eficiencia es crucial.

Microprogramación:

Codificación Horizontal:

Cada microinstrucción contiene tantos bits como señales de control. Cada bit representa una microorden (1 activa, 0 inactiva). Este método permite ejecutar microórdenes simultáneamente pero es ineficiente en términos de memoria ya que la mayoría de los bits estarán en 0. La codificación horizontal ofrece el máximo nivel de **paralelismo** y flexibilidad. Cada señal de control puede ser activada o desactivada independientemente de las demás, permitiendo una gran variedad de combinaciones de microórdenes en una sola microinstrucción. Esto es especialmente útil para operaciones complejas que requieren la activación simultánea de múltiples componentes del sistema.
Sin embargo, esta flexibilidad tiene un costo significativo en términos de uso de memoria. Dado que cada bit corresponde a una señal de control específica, las microinstrucciones tienden a ser muy largas, con muchos bits en estado inactivo (0) en la mayoría de las operaciones. Esto puede llevar a un uso ineficiente del espacio de memoria de control.

Codificación Vertical:

Aprovecha la incompatibilidad entre ciertas microórdenes para reducir el tamaño de la microinstrucción y la memoria de control. Necesita un decodificador que emita las señales correspondientes según la microinstrucción recibida. Aunque es más eficiente en términos de tamaño, el proceso de decodificación es generalmente más lento.
La codificación vertical aborda la ineficiencia de memoria de la codificación horizontal mediante la compresión de la información de control. En lugar de tener un bit para cada señal de control, utiliza una codificación más compacta donde cada microinstrucción representa una operación o conjunto de operaciones específicas.
Esta aproximación **reduce significativamente el tamaño de las microinstrucciones** y, por ende, el tamaño total de la memoria de control. Sin embargo, requiere un paso adicional de decodificación para convertir la microinstrucción codificada en las señales de control correspondientes. Este proceso de decodificación introduce una **latencia adicional** en la ejecución de cada microinstrucción, lo que puede afectar el rendimiento general del sistema.

Codificación por Campo:

Agrupa señales en grupos que no pueden estar activas simultáneamente. Divide la microinstrucción en campos, cada uno correspondiente a un grupo de microórdenes. Lanza una microorden por cada campo, combinando la reducción del tamaño de las microinstrucciones y una codificación directa.
La codificación por campo representa un compromiso entre las codificaciones horizontal y vertical. Aprovecha el hecho de que ciertas señales de control son **mutuamente excluyentes o están relacionadas lógicamente**, y las agrupa en campos dentro de la microinstrucción.
Cada campo puede activar una microorden de su grupo correspondiente. Esto permite una reducción significativa en el tamaño de la microinstrucción en comparación con la codificación horizontal, mientras mantiene un cierto grado de paralelismo y evita la necesidad de un decodificador complejo como en la codificación vertical.
Esta técnica ofrece un buen equilibrio entre eficiencia de memoria y velocidad de ejecución. Permite cierto grado de ejecución simultánea de microórdenes (una por campo) mientras mantiene un tamaño de microinstrucción razonable. Además, la decodificación necesaria es generalmente más simple y rápida que en la codificación vertical pura.

Característica	Codificación Horizontal	Codificación Vertical	Codificación por Campo

Definición	Cada microinstrucción contiene tantos bits como señales de control.	Utiliza una codificación compacta, con un decodificador que traduce la microinstrucción en señales de control.	Agrupa señales en campos, cada uno correspondiente a un grupo de microordenes que no pueden estar activas simultáneamente.
Paralelismo	Máximo nivel de paralelismo, permite ejecutar órdenes simultáneamente.	Menor paralelismo debido al proceso de decodificación.	Permite ejecución simultánea de microordenes dentro de cada campo.
Flexibilidad	Alta, cada señal de control puede ser activada o desactivada.	Menor, operaciones predefinidas que deben ser decodificadas.	Intermedia, permite la activación de microordenes por campo.
Tamaño de Microinstrucciones	Grandes, debido a la cantidad de bits necesarios para cada señal de control.	Cortas, debido a la comprensión de información de control.	Intermedio, cada campo representa un grupo de microordenes.
Complejidad de Decodificación	Sin decodificación adicional, las señales de control se activan directamente.	Requiere un decodificador, los que introducen latencia adicional.	Decodificación simple y rápida, menos compleja que en la codificación vertical.
Velocidad de Ejecución	Muy rápida, sin latencia adicional de decodificación.	Más lenta, debido al proceso de decodificación.	Rápida, con decodificación simple por campo.
Eficiencia de Memoria	Baja, debido a la gran cantidad de bits inactivos.	Alta, tamaño de las microinstrucciones reducido.	Intermedia, en el medio de la vertical y la horizontal.
Ejemplo de Uso	Utilizado en sistemas donde la flexibilidad y el paralelismo son cruciales.	Utilizado en sistemas donde la eficiencia de memoria es crítica y la velocidad no es la máxima prioridad.	Utilizado en sistemas que requieren un equilibrio entre eficiencia de memoria y velocidad de ejecución.

¿Para qué arquitecturas son adecuados los secuenciadores microprogramados?

Los secuenciadores microprogramados son más adecuados para arquitecturas complejas que requieren flexibilidad y capacidad de modificación, como las arquitecturas **CISC** (Complex Instruction Set Computing). Estas arquitecturas se benefician de la capacidad de los **secuenciadores microprogramados** para manejar un conjunto grande y variado de instrucciones. Al almacenar microprogramas en memoria ROM, es posible modificar o añadir nuevas instrucciones sin necesidad de cambiar el hardware, lo cual proporciona una gran flexibilidad. Este enfoque permite a las arquitecturas CISC implementar instrucciones más complejas y poderosas, aunque a costa de una ejecución más lenta en comparación con los secuenciadores cableados.

¿Para qué arquitecturas son adecuados los secuenciadores cableados?

Los secuenciadores cableados son más adecuados para arquitecturas **RISC** (Reduced Instruction Set Computing). Estas arquitecturas se benefician de la velocidad y eficiencia que ofrecen los **secuenciadores cableados**, ya que están diseñadas con un conjunto pequeño de instrucciones simples y específicas. Al utilizar un diseño rígido y optimizado, los secuenciadores cableados pueden ejecutar estas instrucciones de manera muy rápida y eficiente, lo cual es ideal para la filosofía de RISC que prioriza un alto rendimiento y una ejecución rápida. Sin embargo, esta rigidez también implica una menor flexibilidad, ya que cualquier cambio en el conjunto de instrucciones requiere modificaciones en el cableado del secuenciador.

PARALELISMO

El paralelismo se refiere a la capacidad de realizar múltiples operaciones o tareas simultáneamente. Este concepto es fundamental para mejorar el rendimiento y la eficiencia de los sistemas informáticos. El objetivo del paralelismo es reducir el tiempo total de ejecución de un programa al dividirlo en partes que se pueden ejecutar al mismo tiempo, mejorando así la utilización de los recursos del sistema y la velocidad de procesamiento global.

Un sistema fuertemente acoplado es aquel en el que múltiples CPUs interconectadas comparten memoria y un reloj global, permitiendo comunicación rápida y acceso uniforme a los recursos para todas las CPUs.
Un sistema débilmente acoplado es aquel en el que múltiples CPUs independientes, con comunicación limitada generalmente a través de pasaje de mensajes, donde cada procesador tiene su propia memoria local y reloj, sin compartir estos recursos.

Clasificación de Flynn:

Es un esquema propuesto por Michael J. Flynn en 1966 para **clasificar las arquitecturas de computadoras en función de la concurrencia en la secuencia de instrucciones y los caminos de datos**. Esta clasificación es fundamental para entender cómo las diferentes arquitecturas pueden manejar el procesamiento paralelo. Se divide en cuatro categorías principales:

SISD (Single Instruction, Single Data):

En la arquitectura SISD, una única unidad de procesamiento ejecuta **una sola instrucción a la vez y opera sobre un único conjunto de datos**. Este es el modelo de computadora más simple y tradicional, conocido como máquina Von Neumann.

- **No paralelismo:** No explota el paralelismo ni en el flujo de instrucciones ni en el de datos. Cada operación se realiza una tras otra sin simultaneidad.
- **Secuencial:** Las operaciones se realizan secuencialmente, siguiendo un flujo lineal de ejecución.
- **Estructura simple:** Es fácil de diseñar y programar debido a su simplicidad y a la ausencia de mecanismos complejos de paralelismo.
- **Limitaciones de rendimiento:** El rendimiento está limitado por la velocidad de un solo procesador y la tasa de transferencia de datos a través del bus. Esto puede convertirse en un cuello de botella en aplicaciones que requieren procesamiento intensivo.
- **Ejemplo:** Una PC clásica con un único procesador que ejecuta un conjunto de instrucciones secuencialmente.

Este modelo es adecuado para tareas que no requieren procesamiento paralelo y es la base de muchos sistemas computacionales tradicionales. Sin embargo, su rendimiento puede ser insuficiente para aplicaciones modernas que demandan mayor velocidad y eficiencia.

SIMD (Single Instruction, Multiple Data):

En la arquitectura SIMD, una **única instrucción es emitida para operar simultáneamente sobre múltiples conjuntos de datos**. Esto es especialmente útil en **aplicaciones donde se realizan las mismas operaciones sobre grandes cantidades de datos**, como en gráficos y procesamiento de señales.

- **Paralelismo de datos:** Explota el paralelismo a nivel de datos.
- **Sincronización:** Todos los procesadores ejecutan la misma instrucción al mismo tiempo, pero en diferentes datos.
- **Eficiencia:** Muy eficiente para operaciones vectoriales y matriciales.
- **Limitaciones:** La flexibilidad es menor ya que todos los elementos procesan la misma instrucción simultáneamente.
- **Ejemplo:** Matrices de procesadores, GPU (Unidad de Procesamiento Gráfico) para procesamiento gráfico y máquinas vectoriales.

Esta arquitectura es ideal para aplicaciones que pueden aprovechar el paralelismo de datos para mejorar significativamente el rendimiento, especialmente en tareas intensivas en cálculo donde la misma operación debe aplicarse repetidamente a grandes conjuntos de datos.

MISD (Multiple Instruction, Single Data):

En la arquitectura MISD, **múltiples unidades de procesamiento ejecutan diferentes instrucciones simultáneamente sobre un único conjunto de datos**. Esta configuración es rara y poco implementada debido a su complejidad y limitadas aplicaciones prácticas.

- **Paralelismo de instrucciones:** Diferentes instrucciones se aplican al mismo dato**.**

- **Aplicaciones específicas:** Usada en sistemas de alta fiabilidad donde los datos son procesados de **múltiples maneras para obtener redundancia en los resultados.**
- **Complejidad:** Es difícil de implementar y no ofrece claras ventajas en la mayoría de las aplicaciones.
- **Ejemplo:** Algunos sistemas de tolerancia a fallos y procesamiento de señales.

Esta arquitectura es ideal para entornos que requieren una alta fiabilidad y tolerancia a fallos, aunque su complejidad limita su uso en aplicaciones más comunes.

MIMD (Multiple Instruction, Multiple Data):

En la arquitectura MIMD, **múltiples procesadores independientes ejecutan diferentes instrucciones sobre diferentes conjuntos de datos simultáneamente.** Este es el modelo más general y flexible, utilizado en la mayoría de los sistemas multiprocesadores y computadoras paralelas modernas.

- **Paralelismo total:** Explota tanto el paralelismo de datos como el de instrucciones.
- **Flexibilidad:** Cada procesador puede ejecutar su propio programa y operar sobre sus propios datos.
- **Arquitecturas de memoria:** Puede tener memoria compartida (multiprocesadores) o memoria distribuida (multicomputadoras).
- **Eficiencia:** Muy eficiente para tareas que pueden ser divididas en sub-tareas independientes.
- **Ejemplo:** Supercomputadoras modernas, sistemas multiprocesadores como servidores de alto rendimiento, y clusters de computadoras.

Esta arquitectura es ideal para aplicaciones que requieren un alto rendimiento y capacidad de procesamiento paralelo, siendo fundamental en áreas como la computación científica, el análisis de grandes volúmenes de datos y las aplicaciones de inteligencia artificial.

MIMD en Detalle:

Multicomputadoras (Memoria Privada):

Las CPUs (Unidades Centrales de Procesamiento) independientes realizan diferentes trabajos sobre diferentes conjuntos de datos simultáneamente. Cada CPU tiene su propia memoria privada y no comparte memoria directamente con otras CPUs. La comunicación entre CPUs se realiza a través de un sistema de pasaje de mensajes. **(Acoplamiento débil).**

Divisiones:

- **MPP (Massively Parallel Processors - Procesadores Masivamente Paralelos):**
  - **Organización:** Grandes computadoras organizadas en estructuras complejas como cuadrículas o hipercubos.
  - **Uso:** Utilizados en cálculos científicos y de alto rendimiento debido a su **capacidad para manejar grandes volúmenes** de datos y tareas intensivas en cálculo.
  - **Ventajas:**
    - Alta capacidad de procesamiento.
    - Ideal para tareas que requieren un gran poder computacional.
  - **Desventajas:**
    - Muy costosos de construir y mantener.
    - Complejidad en la programación y en la gestión de recursos.

- **COW (Clusters of Workstations - Clusters de Estación de Trabajo):**
  - **Organización:** Conjunto de **computadoras independientes y completas** interconectadas mediante tarjetas de red.
  - **Comunicación:** Los nodos se comunican mediante el pasaje de mensajes.
  - **Ventajas:**
    - Alta escalabilidad, permitiendo la **adición de más nodos según sea necesario**.
    - Tolerancia a fallos, ya que **cada nodo puede trabajar independientemente**.
    - **Costos relativamente bajos** comparados con los MPP, utilizando hardware estándar.

- **Desventajas:**
  - Comunicación más lenta debido a la transferencia de mensajes entre nodos.
  - La eficiencia puede disminuir con la sobrecarga de red y la latencia en la comunicación.

Multiprocesadores (Memoria Compartida):

Las CPUs están **fuertemente acopladas** y trabajan sobre una memoria compartida. Permiten la **comunicación directa** y rápida entre procesadores a través de la memoria compartida. Se divide en SMP (UMA **[Acceso a Memoria Uniforme]**) y NUMA **(Acceso a Memoria No Uniforme)**.

Divisiones:

- **SMP (Symmetric Multiprocessing - Microprocesadores Simétricos):**
  - **Organización:** Dos o más procesadores similares, cada uno con su propia Unidad de Control (UC), Unidad Aritmético-Lógica (ALU), registros, y a veces memoria caché privada.
  - **Interconexión:** Conectados mediante un bus u otro sistema de interconexión, con **tiempos de acceso a la memoria similares para cada CPU.**
  - **Ventajas:**
    - Comunicación y sincronización rápida entre procesadores.
    - Simplicidad en el acceso a datos compartidos.
    - Facilita la programación y la gestión de tareas compartidas.
  - **Desventajas:**
    - Limitación en el número total de procesadores (típicamente menos de 32) debido a la competencia por el bus.
    - Cuello de botella en el acceso a la memoria principal.

- **NUMA (Non-Uniform Memory Access - Acceso a Memoria No Uniforme):**
  - **Organización:** Cada procesador tiene acceso a una memoria local más rápida que la memoria compartida con otros procesadores.
  - **Ventajas:**
    - Reduce el cuello de botella en el acceso a la memoria principal.
    - Mejora el rendimiento al minimizar la latencia de acceso a la memoria local.
  - **Desventajas:**
    - Complejidad en la gestión de la memoria, ya que se deben manejar diferentes **tiempos de acceso a la memoria**.
    - **Coherencia de caché** más compleja, requiriendo **protocolos avanzados** para mantener la consistencia entre las copias de datos en diferentes cachés.

	Multicomputadoras	Multiprocesadores
Memoria	Usan memoria privada y se comunican a través de mensajes	Usan memoria compartida
Escalabilidad	Alta escalabilidad debido a la independencia de nodos	Limitada escalabilidad por el cuello de botella en el acceso a la memoria.
Comunicación	Más lenta debido a la transferencia de mensajes.	Más rápido debido a la memoria compartida.
Aplicaciones	Cálculos científicos y aplicaciones de alto rendimiento que requieren alta escalabilidad.	Servidores y sistemas de bases de datos que requieren acceso rápido y simultáneo a grandes cantidades de datos compartidos.

Memoria Privada:

Son CPUs **débilmente acoplados,** se comunican a través de mensajes por lo que la comunicación es lenta. También conocida como **memoria distribuida**, en los sistemas de memoria privada cada procesador o nodo de computación

tiene su propia memoria local exclusiva. Los procesadores no pueden acceder directamente a la memoria de otros procesadores.

- **Espacios de direcciones separados:** Cada procesador tiene su propio espacio de direcciones de memoria independiente.
- **Comunicación explícita:** Los procesadores deben intercambiar datos mediante el envío y recepción de mensajes a través de una **red de interconexión.**
- **Paso de mensajes:** La comunicación y sincronización entre procesos se realiza mediante protocolos de paso de mensajes como MPI (Message Passing Interface).
- **Escalabilidad:** Es más fácil añadir nuevos nodos o procesadores al sistema sin afectar significativamente el rendimiento global.
- **Localidad de datos:** Cada procesador trabaja principalmente con datos en su memoria local, lo que puede mejorar la eficiencia en ciertos tipos de aplicaciones.
- **Heterogeneidad:** Los sistemas pueden componerse de nodos con diferentes arquitecturas o capacidades, siempre que puedan comunicarse mediante un protocolo común.

Memoria Compartida:

Son CPUs **fuertemente acoplados**, tienen una alta capacidad de comunicación. En los sistemas de memoria compartida, múltiples procesadores o núcleos de procesamiento comparten un espacio de memoria común y unificado. Este modelo de memoria permite que todos los procesadores acceden y manipulen **los mismos datos** en una única área de memoria global.

- **Espacio de direcciones único:** Todos los procesadores ven y acceden al mismo espacio de direcciones de memoria.
- **Acceso directo y uniforme:** Cualquier procesador puede leer o escribir en cualquier ubicación de memoria sin necesidad de comunicación explícita con otros procesadores.
- **Coherencia de caché:** Se requieren mecanismos para mantener la consistencia de los datos entre las cachés locales de los procesadores y la memoria principal compartida.
- **Sincronización implícita:** La comunicación entre procesos se realiza a través de variables compartidas en la memoria común.
- **Latencia de acceso uniforme o no uniforme:** Dependiendo de la arquitectura, el tiempo de acceso a la memoria puede ser igual para todos los procesadores (UMA - Uniform Memory Access) o variar según la ubicación física de la memoria (NUMA - Non-Uniform Memory Access).

Existen 4 casos de memoria compartida: teniendo en cuenta la memoria física (hardware) o lógica (software).

1. **No comparten ni memoria física, ni lógica:** acoplamiento débil y se comunica por mensajes.
2. **Comparten memoria lógica:** existe una única memoria física, pero se realiza una emulación similar a la memoria virtual en SO, cada CPU tiene memoria propia pero física hay una sola.
3. **Comparten memoria física:** existen varios procesadores y una memoria física que es particionada y se distribuye a cada procesador. La comunicación es lenta, simula envío de mensajes pero a través de un buffer compartido.
4. **Comparten memoria lógica y física**: esta es la solución más rápida ya que la comunicación entre los procesadores es muy eficiente.

<u>Diferencias entre conexionado bus o conmutado:</u>

Interconexión por Bus: Es un conjunto de líneas eléctricas que permite la comunicación entre procesadores, memoria y otros dispositivos en un sistema. Solo una transferencia puede ocurrir a la vez en un bus compartido.

- **Ventajas**:
    - **Fácil reconfiguración:** Es simple añadir o quitar dispositivos sin necesidad de cambios complejos en la estructura de interconexión.
    - **Costo:** Generalmente menos costoso de implementar debido a la simplicidad del diseño.
- **Desventajas**:

- **Bajo ancho de banda:** Dado que sólo una transferencia puede ocurrir a la vez, el bus puede convertirse en un cuello de botella, limitando el rendimiento general del sistema.
- **Esperas de comunicación:** Los dispositivos deben esperar su turno para acceder al bus, lo que puede introducir latencias significativas.

- **Variantes:**
  - **Jerarquía de buses:** Para mejorar el rendimiento, se puede utilizar una jerarquía de buses que permita la comunicación independiente y simultánea entre diferentes niveles de buses. Esto ayuda a aliviar el cuello de botella en sistemas más grandes y complejos.

Interconexión Conmutada: Utiliza conmutadores y enlaces punto a punto para conectar procesadores con bloques de memoria principal. Este tipo de interconexión puede reservar caminos completos desde el origen al destino antes de enviar los datos.

- **Ventajas:**
  - **No bloqueante:** Permite múltiples transferencias simultáneas sin que unas bloqueen a otras, lo que mejora significativamente el rendimiento.
  - **Escalabilidad:** Es más fácil de escalar para grandes sistemas, ya que se pueden añadir más conmutadores y enlaces punto a punto para manejar incrementos en la carga de trabajo.
- **Desventajas:**
  - **Complejidad y costo:** Requiere una gran cantidad de conmutadores y enlaces, lo que puede aumentar la complejidad y el costo del sistema.
  - **Configuración:** La configuración y gestión de la interconexión conmutada puede ser más complicada en comparación con un sistema basado en bus.
- **Ejemplo: Conmutación de circuitos:** Este método reserva un camino completo desde el origen al destino antes de enviar un paquete de datos, garantizando así que los datos pueden transmitirse sin interrupciones una vez establecido el camino. Este enfoque es común en redes telefónicas y algunos tipos de redes de computadoras.

	Bus	Conmutado
Estructura	Un único canal compartido para todas las transferencias	Múltiples caminos punto a punto gestionados por conmutadores.
Rendimiento	Limitado por el ancho de banda del canal compartido y el tiempo de espera.	Permite transferencias simultáneas y es no bloqueante.
Escalabilidad	Menos escalable debido a la limitación del canal compartido.	Más escalable con la adición de más conmutadores y enlaces.
Costo y complejidad	Generalmente menos costoso y más simple.	Más costoso y complejo debido a la cantidad de conmutadores y enlaces requeridos.

<u>Comunicación de los multiprocesadores:</u>

Memoria compartida:

- **Interconexión por bus:** Existen 4 diseños.

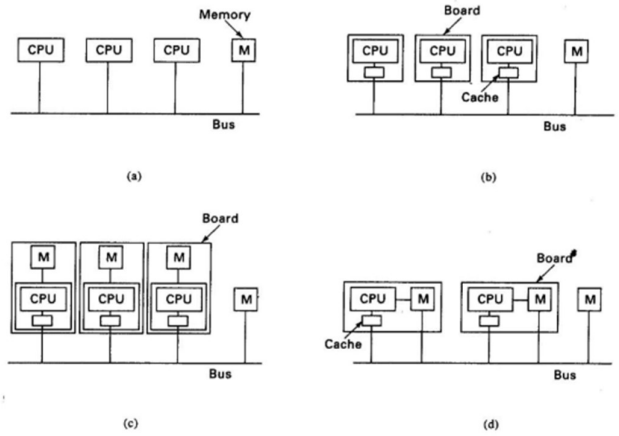


Fig. 8-38. Four designs for implementing shared memory over a single bus.

a) **Diseño básico de memoria compartida**. Múltiples CPUs independientes comparten una memoria principal común, todas conectadas a través de un único bus. Esta configuración es simple pero puede ser ineficiente debido a la contención del bus, ya que todas las CPUs deben competir por el acceso a la memoria a través del mismo canal.

b) **Diseño mejorado con cachés**. Cada CPU tiene su propia memoria caché, lo que reduce la necesidad de acceder constantemente a la memoria principal compartida. Esto disminuye la contención del bus y mejora el rendimiento general del sistema, permitiendo a las CPUs acceder rápidamente a datos frecuentemente utilizados.

c) **Diseño modular con CPUs y cachés en placas separadas**. Cada CPU y su caché asociada están ubicadas en una placa (board) individual. La memoria principal sigue siendo compartida y conectada al bus común. Esta configuración puede mejorar la modularidad, facilitando el mantenimiento y la actualización del sistema.

d) **Diseño de memoria distribuida compartida**. Cada CPU tiene su propia memoria local directamente conectada. Estas memorias locales están interconectadas a través del bus común, creando un espacio de direcciones global virtual. Este diseño permite un acceso rápido a la memoria local de cada CPU, mientras mantiene la posibilidad de acceder a las memorias de otros procesadores a través del bus compartido, ofreciendo un balance entre eficiencia y flexibilidad.

➢ **Interconexión conmutada:**

- Surge para solucionar el problema de cuello de botella del bus compartido y mejorar la escalabilidad del sistema cuando éste crece.
- Se divide la memoria principal en múltiples módulos (por ejemplo, cuatro), creando así varios caminos independientes para el acceso a memoria. Esto reduce el tráfico de cargas en un factor proporcional al número de módulos.
- La conexión entre un procesador y un módulo de memoria no se produce mediante un bus compartido, sino a través de un enlace de punto a punto.
- Existe un conmutador por cada módulo de memoria, que conecta distintos procesadores con la memoria principal. Cuando el conmutador recibe una señal de algún procesador, envía señales a otros procesadores conectados al mismo, indicándoles cuál es el dato que se está solicitando o modificando. Esto es fundamental para manejar la coherencia de caché.
- En un momento dado, solo un procesador puede acceder a un bloque específico de memoria, pero es posible acceder a múltiples módulos de memoria simultáneamente.
- La conexión puede ser de dos tipos:
  - **No bloqueante:** Siempre puede acceder a un destino desocupado. Esto requiere una gran cantidad de conmutadores o "llaves" en la red de interconexión.
  - **Bloqueante:** No siempre puede acceder a un destino desocupado, ya que el camino puede estar ocupado. Esto se debe a que en este caso se cuenta con menos conmutadores o "llaves" en la red.

Memoria privada:

- ➢ **Interconexión por bus:** Los procesadores están organizados en una estructura matricial, donde cada uno posee su propia memoria local con acceso exclusivo. La comunicación entre procesadores se realiza mediante el envío de mensajes, lo que puede resultar en una comunicación más lenta en comparación con los sistemas de memoria compartida. Para compartir datos, es necesario enviar mensajes explícitos a través del bus. Aunque esta configuración puede ofrecer una mayor escalabilidad que los sistemas de memoria compartida, también introduce una mayor complejidad en la programación y potencial latencia en la comunicación.
- ➢ **Interconexión conmutada:** Los procesadores se conectan a través de una red conmutada en lugar de una estructura matricial. Este sistema utiliza conmutadores o **switches** para dirigir los mensajes entre procesadores, ofreciendo mayor flexibilidad y rendimiento en la comunicación en comparación con la interconexión por bus. Permite múltiples comunicaciones simultáneas entre diferentes pares de procesadores, lo que mejora el ancho de banda total del sistema y reduce la contención y los cuellos de botella típicos en sistemas basados en bus. Las topologías comunes en estos sistemas incluyen redes en malla, toro o hipercubo, adaptándose a las necesidades específicas del sistema. Aunque mantiene un sistema de memoria privada, la red conmutada puede ofrecer latencias de comunicación más bajas y un mayor throughput.

COHERENCIA DE CACHÉ

Problema de Coherencia de Caché:

La coherencia de caché se refiere al problema que surge cuando existen múltiples copias del mismo dato en diferentes cachés, y los procesadores actualizan sus copias, lo que puede producir una visión inconsistente de la memoria.
Según las dos políticas de escritura de caché, **Write-Back (WB)** y **Write-Through (WT)**, la política WB puede causar problemas de coherencia. Si dos cachés contienen la misma línea de datos y una de ellas la actualiza, la otra puede operar con un dato desactualizado. Aunque este problema también puede ocurrir con WT, **se puede mitigar si las cachés reciben notificaciones de escritura en la memoria.** Es posible usar WT con modificaciones para mejorar la coherencia.
Existen dos tipos de soluciones para el problema de coherencia de caché: basadas en software y basadas en hardware.

Soluciones Basadas en Software:

Las soluciones de software delegan el problema al compilador y al sistema operativo. Estos clasifican los datos como compartidos o no compartidos, y de solo lectura o de lectura-escritura.

- **Datos no compartidos:** No presentan problemas de coherencia ya que son de uso exclusivo del procesador.
- **Datos de solo lectura:** Tampoco presentan problemas, ya que no se modifican.

El problema surge con los datos compartidos y de escritura. La solución es marcarlos como no-cacheables, evitando que se almacenen en la caché.

Soluciones Basadas en Hardware:

Las soluciones de hardware se denominan protocolos de coherencia de caché. Estas soluciones permiten **reconocer dinámicamente**, en el **momento de la ejecución**, las situaciones potenciales de inconsistencia. A diferencia de las soluciones basadas en software, estos protocolos gestionan el problema cuando realmente ocurre, optimizando el uso de la caché.
Existen dos grandes grupos de protocolos: de directorio y de sondeo.

1. **Protocolos de Directorio:** Recogen y mantienen información sobre dónde residen las copias de las líneas de datos. El directorio centralizado contiene el estado global de los contenidos de las diferentes cachés locales.
    - **Ventajas:** Control centralizado facilita la gestión.

- **Desventajas:** Cuello de botella en el controlador central y costos de comunicación entre el controlador central y las cachés.

2. **Protocolos de Sondeo:** Distribuyen la responsabilidad de mantener la coherencia de caché entre todos los controladores de caché del multiprocesador. **Cada caché debe reconocer cuándo posee una línea que está siendo compartida con otras cachés**. **Cuando una caché modifica una línea, lo anuncia a las demás** mediante un mecanismo de difusión (broadcasting). Cada controlador de caché puede sondear la red para observar las notificaciones y reaccionar rápidamente.
    - **Ventajas:** Distribución de la carga de mantenimiento de coherencia y rápida reacción a las modificaciones.
    - **Desventajas:** Requiere un mecanismo eficiente de broadcasting.

Protocolo MESI: (MOESI → O = Ocupado)

Un ejemplo de protocolo de coherencia de caché es el protocolo MESI, donde cada bloque de la caché puede estar en uno de los siguientes estados:

- **Modificado (M):** El bloque ha sido modificado y **no es consistente con la memoria principal.**
- **Exclusivo (E):** El bloque está presente **solo en esta caché** y es consistente con la memoria principal.
- **Compartido (S):** El bloque está presente en **múltiples cachés** y es consistente con la memoria principal.
- **Inválido (I):** El bloque no contiene datos válidos.

Este protocolo mejora la eficiencia y la coherencia en sistemas multiprocesador, manejando de manera efectiva las actualizaciones y los estados de los bloques en las cachés.

MICROCONTROLADORES

Un **microcontrolador** es un circuito integrado programable que ejecuta órdenes grabadas en su memoria. Es un **sistema cerrado** que incluye CPU, memoria y periféricos en **un solo chip**. Está diseñado para **aplicaciones específicas** y control de dispositivos. Los microcontroladores son comunes en sistemas embebidos, como electrodomésticos, automóviles, dispositivos médicos y sistemas de automatización industrial.

Un **microprocesador** es un circuito integrado que **contiene la CPU de una computadora**. Es un **sistema abierto** que requiere **componentes** externos, como memoria y periféricos, **para funcionar**. Los microprocesadores se utilizan en una amplia gama de dispositivos, incluidos computadores personales, servidores, dispositivos móviles y sistemas embebidos de alto rendimiento.

	Microprocesador	Microcontrolador
Tipo de sistema	Abierto	Cerrado
Buses	Accesibles desde el exterior	Inaccesibles desde el exterior
Memoria	Externa, necesita componentes adicionales	Interna, incluye RAM, ROM y almacenamiento.
Flexibilidad	Alta, se puede utilizar una amplia variedad de aplicaciones.	limitada a aplicaciones específicas y tareas predefinidas.
Complejidad del sistema	Mayor, requiere componentes externos como memoria y periféricos.	Menor, integrado en un solo chip
Costo	Generalmente más caro, debido a la necesidad de componentes adicionales.	Generalmente más barato, todo en un solo chip
Ejemplos de uso	Computadoras, servidores, dispositivos móviles.	Electrodomésticos, automóviles, sistemas de automatización.

Capacidad de procesamiento	Generalmente más potentes y rápidos.	Menos potentes, optimizados para eficiencia y tareas específicas.

Arquitectura Harvard

La arquitectura Harvard utiliza **dos memorias separadas para instrucciones y datos**, permitiendo acceso simultáneo a ambas, lo cual puede incrementar el rendimiento. Esta arquitectura es común en **microcontroladores**, especialmente en aplicaciones donde se requiere un rendimiento elevado y un control preciso, como en sistemas embebidos y dispositivos de tiempo real.
En el diseño de la arquitectura Harvard, la CPU está conectada a dos memorias independientes y con buses diferentes: una para las instrucciones y otra para los datos. Esta configuración permite el **paralelismo**, ya que se puede acceder a los datos e instrucciones de forma independiente y simultánea. Las memorias no necesariamente deben compartir características, por lo que pueden tener distinta longitud de datos e instrucciones, permitiendo una optimización del uso de la memoria en general.
El tiempo de acceso a las instrucciones puede superponerse con el de los datos, aumentando la velocidad de procesamiento en cada operación. Además, facilita la realización de **pipeline**, una técnica de segmentación que permite al procesador ejecutar una instrucción mientras busca el código de la siguiente. Las instrucciones y los datos se almacenan en caches separadas para mejorar el rendimiento. Sin embargo, esto tiene la desventaja de dividir la cantidad de cache entre los dos tipos de información, lo que solo funciona mejor cuando la **frecuencia de lectura de instrucciones y de datos es aproximadamente la misma**. Además, se requieren dos buses separados, lo que hace que la arquitectura sea más compleja de manufacturar y más cara.

Arquitectura von Neumann

La arquitectura de Von Neumann utiliza una sola memoria para almacenar tanto las instrucciones como los datos. Esta arquitectura describe a la computadora con **cuatro secciones principales: una memoria principal, dispositivos de entrada-salida, la Unidad Central de Procesamiento (CPU) y las interconexiones o buses entre todos los bloques funcionales del sistema**. Es común en **microprocesadores** y procesadores de señal digital, debido a su simplicidad y flexibilidad, lo que facilita el diseño de sistemas de propósito general, como los que se encuentran en las computadoras personales.
En este diseño, la CPU está conectada a **una memoria principal única** (generalmente RAM), donde se guardan tanto las instrucciones del programa como los datos, lo cual hace que el procesamiento sea eficiente debido a que todo está ubicado en un solo lugar. Esto elimina la necesidad de buses separados y permite una búsqueda más directa y eficiente. Sin embargo, **requiere un mecanismo para distinguir entre datos e instrucciones, ya que se almacenan en el mismo formato.**
La arquitectura de Von Neumann es relativamente fácil de diseñar y eficiente en el uso de la memoria, ya que no desperdicia espacio de almacenamiento al ser usada por ambos tipos de información. Una particularidad de esta arquitectura es que **la conexión se realiza mediante un único bus.** El tamaño de las instrucciones está fijado por el ancho del bus; por ejemplo, un microprocesador de 8 bits con un bus de 8 bits tendrá que manejar datos e instrucciones en unidades de 8 bits (bytes). Si se necesita acceder a una instrucción que excede el ancho del bus, se deberán realizar múltiples accesos a la memoria.
Tener un único bus hace más lenta la respuesta del microprocesador, ya que no puede buscar en memoria una nueva instrucción mientras está transfiriendo datos de la instrucción anterior. Además, **requiere de dos ciclos de reloj para leer la memoria, uno para las instrucciones y otro para los datos.** El cuello de botella de Von Neumann se puede mejorar de varias maneras, como con el uso de memoria caché entre la CPU y la memoria principal, o con vías independientes para instrucciones y datos (arquitectura Harvard modificada), para reducir el acceso a memoria.

Conjunto de instrucciones:

Los **microprocesadores** suelen ser **CISC**, estos tienen un conjunto de instrucciones grande y complejo, diseñado para realizar tareas complejas en pocas líneas de código. **(Secuenciadores microprogramados)**

- **Características:**
  - **Conjunto de instrucciones grande:** CISC incluye muchas instrucciones, algunas de las cuales pueden realizar tareas muy complejas.
  - **Complejidad:** Las instrucciones complejas pueden ejecutar múltiples operaciones en una sola instrucción, lo que reduce la cantidad de líneas de código necesarias para una tarea.
  - **Eficiencia en codificación:** Diseñado para minimizar la cantidad de instrucciones en un programa, lo que puede reducir el tamaño del código fuente.

    - **Variedad de modos de direccionamiento:** CISC soporta múltiples modos de direccionamiento, lo que facilita la programación de tareas complejas.
    - **Mayor consumo de energía:** La complejidad de las instrucciones y la necesidad de decodificarlas puede llevar a un mayor consumo de energía y una menor eficiencia.
- **Aplicaciones:** Utilizados en computadoras personales, servidores y estaciones de trabajo donde la flexibilidad y la capacidad de realizar tareas complejas son cruciales.

Los **microcontroladores** suelen ser **RISC**, estos tienen un conjunto de instrucciones más pequeño y simplificado, optimizado para realizar operaciones básicas de manera rápida y eficiente. Esto es beneficioso para aplicaciones embebidas que requieren alta eficiencia y bajo consumo de energía.

- **Características:**
    - **Conjunto de instrucciones pequeño:** RISC incluye un número limitado de instrucciones, cada una de las cuales está diseñada para realizar una **operación muy básica**.
    - **Simplicidad:** Las instrucciones simples permiten que cada una sea ejecutada rápidamente, usualmente en **un solo ciclo de reloj**.
    - **Optimización para operaciones básicas:** Está optimizado para realizar operaciones simples de manera rápida y eficiente.
    - **Eficiencia en la ejecución:** La simplicidad del conjunto de instrucciones permite una ejecución más rápida y un menor consumo de energía.
    - **Pipeline:** La arquitectura RISC facilita el uso de técnicas de pipeline, mejorando aún más la eficiencia. En RISC, el pipeline tiene solo 2 etapas principales. Estas son:
        - **Búsqueda de operación**: En esta etapa, se obtiene la instrucción desde la memoria.
        - **Ejecución de la instrucción**: Aquí se realiza la operación indicada por la instrucción.

      Cabe destacar que el análisis de instrucción no es necesario en **RISC porque no hay microprograma**; RISC trabaja con **secuenciadores cableados**. En cuanto al cálculo de la dirección del operando, las **transferencias se hacen entre registros**, y la búsqueda del operando no es necesaria ya que **el operando está en registros de acceso** muy rápido.
      Además, cada instrucción en RISC toma solo **un ciclo de reloj**, lo que contribuye a su alta eficiencia.

- **Aplicaciones:** Ideal para aplicaciones embebidas que requieren alta eficiencia y bajo consumo de energía, como electrodomésticos, automóviles, dispositivos médicos y sistemas de automatización industrial.

Característica	RISC	CISC
Simplicidad de Instrucciones	Instrucciones simples, típicamente **1 ciclo de ejecución**.	Instrucciones complejas, pueden tomar **múltiples ciclos de ejecución**.
Acceso a Memoria	Referencias a memoria solo con instrucciones LOAD/STORE.	Cualquier instrucción puede referenciar a la memoria.
Pipeline	Más eficaz, con menos etapas.	Menos eficaz, con más etapas.
Ejecución de Instrucciones	Ejecutadas directamente por el **hardware**.	Interpretadas por un **microprograma**.
Formato de Instrucciones	Formato **fijo** de instrucciones.	Formato **variable** de instrucciones.
Cantidad de Instrucciones	Pocas instrucciones y modos de instrucción.	Muchas instrucciones y modos.
Complejidad del Sistema	La complejidad se encuentra en el **compilador**.	Complejidad en el **microprograma**.
Registros	Conjunto **múltiple** de registros.	Conjunto **único** de registros.

En **conclusión**, podemos decir que las máquinas RISC completan todas sus instrucciones en un solo ciclo de reloj, es por eso que es más rápida que las máquinas CISC, y en RISC contamos con un gran manejo de registros. Sin embargo,

para efectuar una operación de mayor complejidad (como multiplicaciones o divisiones por ejemplo) necesitamos más instrucciones como en CISC.
Por eso un usuario debe definirse a favor o en contra de una arquitectura en función de la aplicación que desea realizar.

PIPELINE en los microcontroladores:

El **pipeline** en los **microcontroladores** es una **técnica segmentada** utilizada para aumentar la eficiencia y la velocidad de procesamiento al dividir la **ejecución de una tarea en múltiples etapas independientes**. **Estas etapas pueden ejecutarse simultáneamente en diferentes partes del hardware del microcontrolador**. La tarea de ejecutar una instrucción se divide en varias etapas, como la búsqueda de la instrucción (fetch), la decodificación (decode), la ejecución (execute), la memoria (memory), y la escritura del resultado (write-back).
Cada una de estas etapas puede trabajar en diferentes instrucciones al mismo tiempo. Por ejemplo, mientras una instrucción está siendo decodificada, otra puede estar siendo buscada y otra puede estar siendo ejecutada. Sin embargo, las dependencias de datos y otros problemas pueden causar ralentizaciones, por lo que se requieren diversas estrategias para mitigar estos problemas y mantener el flujo de instrucciones lo más fluido posible.

Las estrategias para mitigar problemas en el pipeline incluyen:

- **Interbloqueo (Stalling)**: Detener temporalmente el pipeline hasta que se resuelva la dependencia de datos.
- **Reordenamiento de instrucciones**: Cambiar el orden de las instrucciones para evitar conflictos.
- **Bypassing/Forwarding**: Usar el resultado de una instrucción antes de que sea escrito de nuevo en los registros, pasando directamente al siguiente paso que lo necesita.
- **Predicción de saltos**: Anticipar el resultado de una instrucción de salto para mantener el pipeline lleno y evitar interrupciones.

Ventajas:

- **Mayor Rendimiento**: Aumenta la velocidad de ejecución al permitir que diferentes etapas del procesamiento de instrucciones se realicen en paralelo.
- **Eficiencia**: Reduce el tiempo de ciclo por instrucción al superponer la ejecución de múltiples instrucciones.

Desventajas:

- **Interbloqueos**: Puede enfrentar problemas de interbloqueo (stalls) cuando hay dependencias de datos entre instrucciones consecutivas.
- **Complejidad de Control:** Requiere mecanismos adicionales para manejar dependencias y predicciones de saltos.

Comparación con Microprocesadores:
Los **microprocesadores**, diseñados para tareas más complejas y de alto rendimiento, implementan pipeline de manera más agresiva y sofisticada. Tienen más recursos disponibles y pueden permitirse un pipeline más profundo y complejo, lo cual mejora significativamente el rendimiento a costa de mayor consumo de energía.
Por otro lado, los **microcontroladores** están diseñados para aplicaciones específicas y de menor consumo energético, por lo que su implementación de pipeline tiende a ser más sencilla y menos profunda.

Comparación con Procesador Sin Pipeline:

- **Sin Pipeline**: Cada instrucción debe completarse antes de que la siguiente comience, lo que puede resultar en una ejecución más lenta.
- **Ventaja de Sin Pipeline**: Diseño más sencillo y menor complejidad en la unidad de control.

Formato de instrucciones ortogonal:

El **formato de instrucciones ortogonal** es un diseño de conjunto de instrucciones en el que cualquier operación puede ser realizada con cualquier registro o modo de direccionamiento disponible. Esto proporciona una gran flexibilidad y simplicidad en la programación, ya que permite utilizar cualquier combinación de registros y modos de direccionamiento para cualquier instrucción. Sin embargo, este formato puede resultar en instrucciones de mayor tamaño, lo que aumenta la complejidad de decodificación, y también incrementa la complejidad del hardware al requerir soporte para todas las combinaciones posibles.

Ventajas:

- Flexibilidad: Facilita la programación al permitir una mayor flexibilidad en el uso de registros y modos de direccionamiento.

- Simplicidad: Simplifica el diseño del compilador y el código ensamblador.

Desventajas:

- Tamaño de Instrucciones: Puede resultar en instrucciones de mayor tamaño, lo que aumenta la complejidad de decodificación.
- Costo: Aumenta la complejidad del hardware, ya que se necesita soporte para todas las combinaciones posibles de registros y modos de direccionamiento.

Amplio banco de registros:

Un **amplio banco de registros** se refiere a disponer de una gran cantidad de registros accesibles por el procesador para operaciones rápidas. Esto reduce la necesidad de acceder a la memoria principal, acelerando el procesamiento y permitiendo que más variables temporales y datos intermedios se almacenan en registros, optimizando la eficiencia del código. Sin embargo, tener más registros puede complicar la gestión de contexto durante interrupciones o cambios de tarea, y aumentar el costo y la complejidad del hardware en comparación con un banco de registros limitado, que requiere más acceso a memoria y puede ralentizar el procesamiento, aunque simplifica el diseño y reduce los costos.

Ventajas:

- Eficiencia: Reduce la necesidad de acceder a la memoria principal, lo que acelera el procesamiento.
- Optimización de Código: Permite que más variables temporales y datos intermedios se almacenan en registros, mejorando la eficiencia del código.

Desventajas:

- Complejidad de Manejo: Un mayor número de registros puede complicar la gestión de contexto durante interrupciones o cambios de tarea.
- Costo de Hardware: Más registros pueden aumentar el costo y la complejidad del hardware.

Comparación con Registros Limitados:

- Registros Limitados: Menos registros requieren más acceso a memoria, lo que puede ralentizar el procesamiento.
- Ventaja de Registros Limitados: Diseño más sencillo y menor costo de hardware.

PIPELINE

Es una técnica de diseño en la arquitectura de computadoras que busca mejorar el rendimiento del procesador mediante la ejecución paralela de múltiples instrucciones en diferentes etapas de procesamiento. La ejecución de una instrucción se divide en varias etapas, y múltiples instrucciones pueden estar en diferentes etapas simultáneamente, lo que permite que el procesador trabaje en varias instrucciones al mismo tiempo.

Beneficios:

- **Aumento del rendimiento**: incrementa el número de instrucciones que se pueden ejecutar por unidad de tiempo.
- **Paralelismo implícito**: Cada etapa del pipeline trabaja en una instrucción diferente simultáneamente.
- **Aprovechamiento de recursos**: Permite un uso más eficiente de los recursos del procesador.
- **Reducción del tiempo de ejecución promedio**: Reduce el tiempo promedio por instrucción en un flujo continuo de instrucciones.
- **Frecuencia de reloj más alta**: Al dividir el trabajo en etapas más pequeñas, cada etapa puede completarse más rápidamente, permitiendo una frecuencia de reloj potencialmente más alta.

Aplicaciones:

- **Procesadores de Alto Rendimiento:** Utilizado ampliamente en procesadores modernos para mejorar el rendimiento en aplicaciones de alta demanda, como gráficos, juegos, procesamiento de datos y servidores.
- **Microcontroladores Avanzados:** Algunos microcontroladores también incorporan pipelines para mejorar su eficiencia en aplicaciones embebidas críticas.

Instrucciones que generan problemas:

Las instrucciones de salto (condicionales e incondicionales), que ocupan aproximadamente el 30% de las instrucciones totales. Si se efectúa un salto, las etapas que se hayan ejecutado de las instrucciones siguientes del pipeline no sirven y se deben limpiar todos los registros.

Soluciones:

- **Dejar de cargar el pipeline hasta atender el salto**. Esto no es muy efectivo debido a que la espera tarda mucho tiempo y no se hace nada durante la misma.
- Otra alternativa es **tratar al salto como una instrucción cualquiera** y seguir cargando el pipeline. Si el salto es exitoso, se limpia el pipeline y se carga a partir de la posición a la que lleva el salto. Sin embargo, tampoco es muy efectiva, debido a que la limpieza del pipeline demanda mucho tiempo. Limpiar consiste en volver a cargar el valor de los registros que las instrucciones hayan modificado; para esto se necesita guardar los valores previos.
- **Predicción de saltos**: Consiste en predecir los saltos anotando el comportamiento del programa en saltos anteriores. Puede ser estática o dinámica.
    - Estática: El compilador hace una suposición por cada una de las instrucciones de salto que genera.
    - Dinámica: En tiempo de ejecución, el microprograma constituye una tabla con las direcciones que contienen saltos y guarda un registro de comportamientos de cada uno (si se efectuó o no). Se asocian dos o más bits a cada instrucción de salto, que actúan como un contador: si el salto se efectúa se suma, si no se efectuó se resta.

PIPELINE genérico de cinco etapas de una máquina-CISC:

1. **Fetch**: Obtener la instrucción de la memoria. Se obtiene la instrucción de la memoria de instrucciones y se carga en el registro de instrucciones.
2. **Decode**: Decodificar la instrucción y leer los registros fuente. La instrucción se decodifica para determinar qué operación se va a realizar, y se leen los registros fuente necesarios.
3. **Execute**: Realizar la operación aritmética o lógica (ALU). Se realiza la operación aritmética o lógica especificada por la instrucción.
4. **Memory access:** Acceder a la memoria para leer o escribir datos. Si la instrucción es una operación de carga o almacenamiento, se accede a la memoria para leer o escribir datos.
5. **Write back**: Escribir el resultado en los registros. El resultado de la operación se escribe en el registro de destino.

Cada etapa del pipeline puede procesar una instrucción diferente simultáneamente. Esto significa que mientras una instrucción está en la etapa de decodificación, otra puede estar en la etapa de búsqueda y otra en la etapa de ejecución. De esta manera, múltiples instrucciones pueden estar en diferentes etapas del pipeline al mismo tiempo, permitiendo que el procesador trabaje en varias instrucciones simultáneamente.

Diagrama de ejecución de PIPELINE de 5 tiempo indicando la instrucciones en ese etapas:

Ciclo	1	2	3	4	5	6	7	8	9
Fetch	A	B	C	D	E	F	G	H	I
Decode		A	B	C	D	E	F	G	H
Execute			A	B	C	D	E	F	G
Memory access				A	B	C	D	E	F
Write back					A	B	C	D	E

Explicación:

1. Ciclo 1: Se inicia el fetch de la instrucción A.
2. Ciclo 2: A pasa a decode, mientras B inicia su fetch.

3. Ciclo 3: A pasa a execute, B a decode, C inicia su fetch.
4. Ciclo 4: A pasa a memory access, B a execute, C a decode, D inicia su fetch.
5. Ciclo 5: A llega a write back, B a memory access, C a execute, D a decode, E inicia su fetch.

A partir del ciclo 5, el pipeline está completamente lleno y se están procesando 5 instrucciones simultáneamente, una en cada etapa.

Este diagrama muestra claramente:

1. Las 5 etapas del pipeline CISC (Fetch, Decode, Execute, Memory, Write Back).
2. Cómo múltiples instrucciones (A, B, C, etc.) avanzan por las etapas del pipeline.
3. El paralelismo alcanzado, con una nueva instrucción iniciando cada ciclo.
4. Que a partir del ciclo 5, se completa una instrucción por ciclo (aunque cada instrucción tarda 5 ciclos en completarse).

Pipeline en RISC:

El pipeline en arquitecturas RISC (Reduced Instruction Set Computing) se caracteriza por su simplicidad y eficiencia, consta de solo dos etapas principales:

- Búsqueda del operando: En esta fase, el procesador localiza y obtiene los datos necesarios para ejecutar la instrucción. Estos datos suelen estar en registros, lo que permite un acceso rápido.
- Ejecución de la instrucción: Una vez obtenidos los operandos, el procesador realiza la operación especificada por la instrucción.

A diferencia de las arquitecturas CISC (Complex Instruction Set Computing), RISC no necesariamente utiliza una etapa de análisis de instrucciones compleja. Esto se debe a que, **en muchos casos**, RISC no emplea microprogramación, sino que puede trabajar con secuenciadores cableados, lo que significa que las instrucciones están directamente implementadas en hardware. Las operaciones en RISC se realizan principalmente entre registros, lo que incluye el cálculo de direcciones de operandos y las transferencias de datos. Este enfoque de registro a registro contribuye a la velocidad y eficiencia del pipeline. La arquitectura RISC se beneficia aún más cuando se implementa en una arquitectura Harvard. Esta combinación permite la separación física de las memorias de instrucciones y datos, posibilitando el acceso simultáneo a ambas y aumentando así el rendimiento del pipeline.

Aunque optimizada, la segmentación en RISC sigue presentando ciertos problemas. Anteriormente, vimos que las instrucciones de salto y las lecturas a memoria generan complicaciones al segmentar instrucciones. Para solucionar estos problemas, el compilador o ensamblador puede insertar una operación NOOP en el flujo de instrucciones. La instrucción NOOP, irónicamente, no realiza ninguna operación sobre los datos. En cambio, resulta útil para generar un retardo en el flujo de instrucciones. Esto es necesario cuando, en una operación segmentada, se necesita evaluar una condición de salto o cuando una instrucción depende de un dato que está a punto de ser actualizado, y usualmente se coloca después de que todos los registros involucrados han sido cargados o escritos.

Existe una forma de optimizar el programa mediante estos retardos, específicamente utilizando lo que se conoce como salto retardado y salto retardado optimizado:

- **Salto Retardado:** Consiste en colocar un NOOP después del salto. Esto tiene la ventaja de que, para cuando el flujo del programa alcanza el NOOP, la instrucción de salto ya se está ejecutando y el salto ya se habrá realizado. Además, el programa no tiene que esperar hasta completar una operación de lectura/escritura realizada previamente para saltar, garantizando que no se comience a buscar la operación siguiente al salto antes de tiempo.
- **Salto con Retardo Optimizado**: Hace algo similar, pero reemplaza el NOOP por la instrucción previa al salto, intercambiando sus lugares. Así, primero se evalúa y ejecuta el salto. Dado que la instrucción anterior era necesaria y al mismo tiempo es la próxima a ejecutarse, se garantiza que el salto se realice correctamente y que la instrucción con la que se cambió de lugar ya esté en proceso de búsqueda para su ejecución.

Cabe destacar que este último método sólo se aplica a saltos incondicionales, llamadas y retornos. Para saltos condicionales, el compilador utiliza un salto con retardo normal, insertando el NOOP después del salto. De esta forma, al evaluar la condición del salto, no se realiza ninguna otra operación que comprometa el estado del procesador y de los registros.

MEMORIA CACHÉ (MC)

Es un tipo de memoria de acceso rápido que se encuentra entre la CPU y la memoria principal de una computadora. Su función principal es almacenar temporalmente datos e instrucciones que se utilizan con frecuencia, lo que ayuda a acelerar el acceso a ellos y mejorar el rendimiento del sistema.
El paradigma principal en que se basa el funcionamiento de la memoria caché es el **principio de localidad de referencia**. Este principio se divide en:

- **Localidad temporal:** Los datos a los que se ha accedido recientemente es probable que se vuelvan a necesitar en un futuro cercano.
- **Localidad espacial**: Los datos cercanos en memoria a los que ha accedido recientemente es probable que se necesiten pronto.

La mejora efectiva que se logra usando la memoria caché es la reducción del tiempo promedio de acceso a la memoria, lo que incrementa el rendimiento del sistema. Esto se debe a que la caché almacena temporalmente los datos más frecuentes utilizados, permitiendo que el procesador acceda a estos de manera mucho más rápida que si tuviera que acceder a la memoria principal (RAM) cada vez.
La memoria caché es principalmente una **implementación de hardware.** Sin embargo, su eficacia también puede ser mejorada con estrategias de software que optimicen el uso de la caché, como las técnicas de programación que promueven la localización de referencia.

Función de mapeo:

Es el mecanismo fundamental que determina cómo los bloques de memoria principal se asignan a las líneas de la memoria caché. Esta función es crucial para el funcionamiento eficiente de la memoria caché por las siguientes razones:

1. **Asignación de bloques**: Define cómo se distribuyen los datos de la memoria principal en la estructura más pequeña de la caché.
2. **Optimización del acceso**: Busca equilibrar la velocidad de acceso con la utilización eficiente del espacio en caché.
3. **Gestión de conflictos**: Ayuda a manejar situaciones donde múltiples bloques de memoria principal podrían mapear a la misma ubicación en caché.
4. **Impacto en el rendimiento**: La elección de la función de mapeo afecta directamente la tasa de aciertos (hit rate) de la caché, lo que influye en el rendimiento global del sistema.
5. **Flexibilidad vs. Complejidad:** Diferentes funciones de mapeo ofrecen distintos grados de flexibilidad en la asignación, pero también pueden variar en su complejidad de implementación.

La función de mapeo es esencial para determinar:

- Dónde se almacena un bloque específico de memoria principal dentro de la caché.
- Cómo se localiza un dato cuando el procesador lo solicita.
- Qué estrategia se utiliza para reemplazar datos en la caché cuando está llena.

Existen tres principales tipos de funciones de mapeo:

Mapeo Directo:

El Mapeo Directo es una técnica de función de mapeo en la memoria caché que **asigna cada bloque de la memoria principal a una única ubicación en la caché**. En este esquema, múltiples bloques de memoria principal pueden corresponder a la misma línea de caché, pero solo uno a la vez. Utiliza una fórmula simple, típicamente (Dirección del bloque) mod (Número de líneas en caché), para determinar la ubicación. Cada línea de caché se estructura con una

etiqueta, un índice y un desplazamiento. Para acceder a los datos, el índice localiza la línea, la etiqueta verifica el bloque correcto, y el desplazamiento encuentra el byte específico. Aunque es rápido y fácil de implementar, el Mapeo Directo puede generar conflictos cuando varios bloques compiten por la misma línea, lo que puede resultar en reemplazos frecuentes y una utilización ineficiente del espacio de caché, potencialmente aumentando la tasa de fallos en ciertos patrones de acceso a memoria. Este tipo de función de mapeo no requiere de algoritmos de reemplazo.

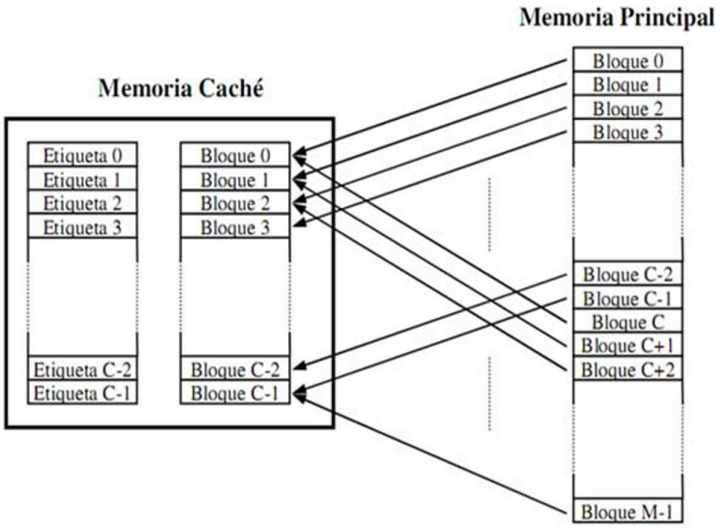


Figura 5.7 Asignación de bloques de la M_p en la M_{ca} con correspondencia directa.

Ventajas:

- Es fácil de implementar
- La búsqueda dentro de la memoria caché es rápida.

Desventajas:

- No se aprovecha la memoria caché al máximo, debido a que se van a reemplazar bloques así sea que esta no está llena.
- Es muy probable que exista competencia por una misma línea en memoria caché, aun cuando esta no esté llena. Esto va a pasar cuando dos o más bloques tengan asignado la misma línea en memoria caché. Lo que termina reduciendo la tasa de aciertos y provocando miss de caché.

Mapeo asociativo:

El Mapeo Asociativo es una técnica de función de mapeo en la memoria caché que **permite que cualquier bloque de la memoria principal pueda ser almacenado en cualquier línea de la caché**. Este enfoque ofrece máxima flexibilidad, eliminando los conflictos de ubicación presentes en el mapeo directo. En este esquema, cada línea de caché contiene tanto los datos como una etiqueta que identifica el bloque de memoria principal correspondiente. Para encontrar un dato, se debe buscar en toda la caché, comparando la etiqueta de cada línea con la dirección solicitada. Esto resulta en un uso más eficiente del espacio de caché y una tasa de aciertos potencialmente mayor, especialmente para patrones de acceso a memoria irregulares. Sin embargo, la búsqueda exhaustiva necesaria para localizar los datos puede aumentar la complejidad del hardware y el tiempo de acceso. El mapeo asociativo es particularmente útil en cachés pequeñas o en situaciones donde la predicción de patrones de acceso a memoria es difícil, pero su implementación es más costosa en términos de hardware comparado con otras técnicas de mapeo.

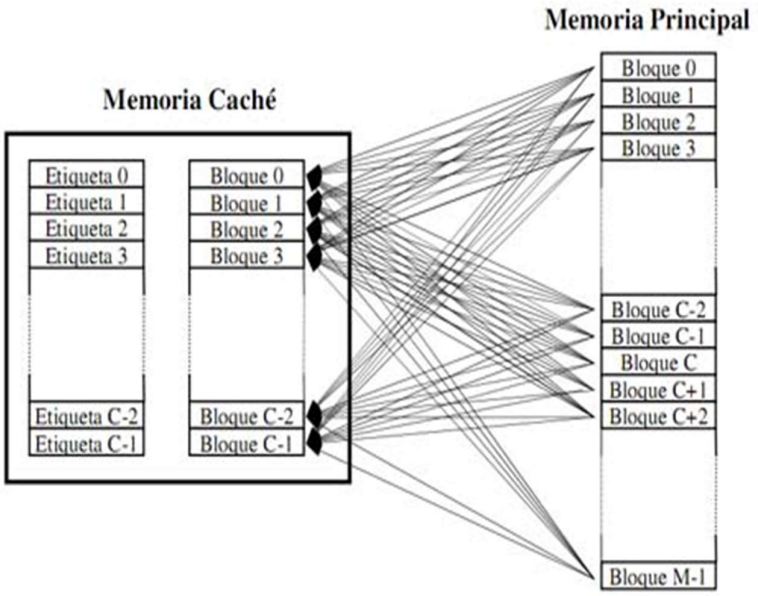


Figura 5.9 Asignación de bloques de la M_p en la M_{ca} con correspondencia totalmente asociativa.

Ventajas:

- Se aprovecha al máximo a la memoria caché, no se reemplaza un bloque de la misma a no ser que esta esté llena.
- Aumenta la tasa de aciertos.

Desventajas:

- La búsqueda en memoria caché es más lenta que en el caso anterior.
- Se debe recorrer la memoria caché comparando el TAG de cada línea con el TAG de la palabra que se busca.

Asociativo por conjunto:

El Mapeo Asociativo por Conjunto es una técnica de administración de la memoria caché que combina las ventajas del mapeo directo y el mapeo totalmente asociativo. Esta estrategia divide la caché en conjuntos, cada uno conteniendo múltiples líneas. Un bloque de memoria principal se asigna a un conjunto específico (como en el mapeo directo), pero puede ocupar cualquier línea dentro de ese conjunto (como en el mapeo asociativo).

Esta técnica ofrece un equilibrio entre velocidad de acceso y tasa de aciertos. La búsqueda se limita a un conjunto específico, lo que es más rápido que una búsqueda completa de la caché, mientras que la flexibilidad dentro del conjunto reduce los conflictos de ubicación.

El **grado de asociatividad** (N) determina cuántas líneas hay en cada conjunto. Cuando N=1, equivale al mapeo directo; cuando N iguala el número total de líneas en la caché, se convierte en mapeo totalmente asociativo. En la práctica, se elige un valor intermedio de N para optimizar el rendimiento, aprovechando tanto la rapidez de búsqueda del mapeo directo como la alta tasa de aciertos del mapeo asociativo.

Esta implementación proporciona un compromiso eficaz entre complejidad de hardware, velocidad de acceso y eficiencia en el uso del espacio de caché, haciéndola una opción popular en muchos sistemas modernos.

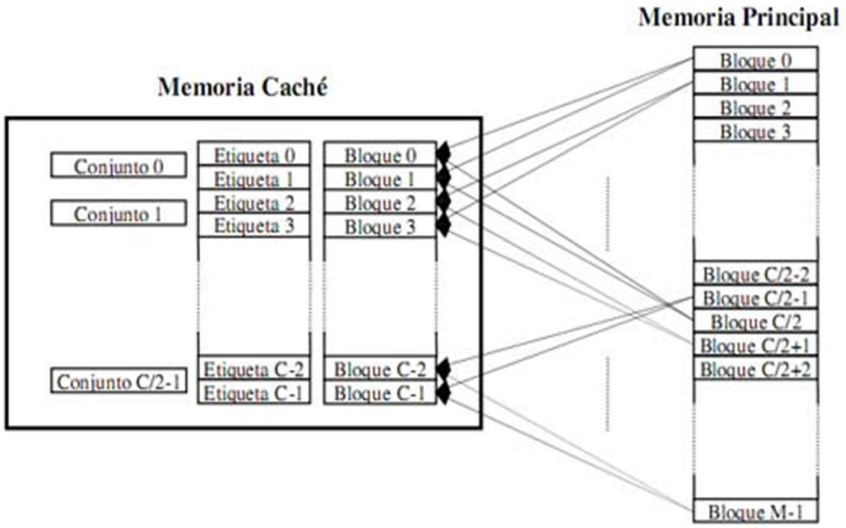


Figura 5.11 Asignación de bloques de la M_p en la M_{ca} con correspondencia asociativa por conjuntos.

Algoritmo de reemplazo:

Es una estrategia crucial en la gestión de la memoria caché que determina qué línea de la caché debe ser reemplazada cuando se necesita almacenar un nuevo bloque y la caché está llena. Este algoritmo es fundamental para mantener la eficiencia y el rendimiento de la memoria caché por las siguientes razones:

1. **Optimización del rendimiento**: Busca mantener en la caché los datos que tienen más probabilidades de ser utilizados en el futuro cercano.
2. **Gestión de conflictos**: Resuelve situaciones donde múltiples bloques compiten por el mismo espacio en la caché.
3. **Balanceo de prioridades**: Equilibra la necesidad de mantener datos **frecuentemente utilizados** con la de incorporar nuevos datos potencialmente importantes.
4. **Impacto en la tasa de aciertos**: La elección del algoritmo afecta directamente la tasa de aciertos de la caché, influyendo así en el rendimiento global del sistema.
5. **Adaptabilidad**: Algunos algoritmos pueden adaptarse a diferentes patrones de acceso a memoria para optimizar su eficacia.
6. **Complejidad vs. Eficacia:** Los algoritmos varían en su complejidad de implementación y en su eficacia para diferentes escenarios de uso.

Los algoritmos de reemplazo consideran factores como:

- Frecuencia de uso de los bloques
- Tiempo transcurrido desde el último acceso
- Patrones de acceso a memoria

Algunos de los algoritmos de reemplazo más comunes incluyen:

FIFO (First In, First Out):

Es un algoritmo de reemplazo en memoria caché que opera bajo el principio de que el primer bloque que entró en la caché será el primero en ser reemplazado cuando se necesite espacio para un nuevo bloque. Este método trata la caché como una cola, donde los bloques más antiguos se encuentran al frente y son los primeros candidatos para ser eliminados.

Funcionamiento: Mantiene un registro del orden en que los bloques fueron cargados en la caché. Cuando se necesita espacio, selecciona para reemplazo el bloque que ha estado en la caché por más tiempo.No considera la frecuencia de uso o el tiempo desde el último acceso a los bloques.

Ventaja:

- Es simple de implementar.
- Tiene un bajo costo computacional.
- Overhead mínimo en términos de procesamiento y almacenamiento.

Desventaja:

- Puede ser ineficiente si los bloques más antiguos son también los más frecuentemente utilizados.
- No se adapta a patrones cambiantes de acceso a memoria.

LRU (Least Recently Used):

Es un algoritmo de reemplazo en memoria caché que opera bajo el principio de reemplazar el bloque que no ha sido accedido durante el período más largo. Este método prioriza mantener en caché los datos que se han utilizado más recientemente, asumiendo que es más probable que se necesiten de nuevo en un futuro cercano.

Funcionamiento: Mantiene un **registro del tiempo** de último acceso para cada bloque en la caché. Cuando se necesita espacio, selecciona para reemplazo el bloque que tiene el tiempo de último acceso más antiguo. Se actualiza este registro cada vez que se accede a un bloque.

Ventajas:

- Tiende a eliminar los bloques menos utilizados, lo que puede mejorar el rendimiento al mantener en caché los datos más relevantes.
- Se adapta bien a patrones de acceso a memoria cambiantes.
- Aprovecha eficazmente el principio de **localidad temporal.**

Desventajas:

- Podría tener un overhead de seguimiento, especialmente en sistemas con un gran número de bloques en la caché y un alto volumen de accesos.
- Requiere hardware más complejo para su implementación comparado con algoritmos más simples como FIFO.
- Puede ser menos efectivo en situaciones donde el patrón de acceso no sigue el principio de localidad temporal.

LFU (Least Frequently Used):

Es un algoritmo de reemplazo en memoria caché que opera bajo el principio de reemplazar el bloque que ha sido accedido con menor frecuencia. Este método prioriza mantener en caché los datos que se han utilizado más a menudo, asumiendo que los bloques frecuentemente accedidos tienen más probabilidades de ser necesarios en el futuro.

Funcionamiento: Mantiene un **contador de accesos** para cada bloque en la caché. Cuando se necesita espacio, selecciona para reemplazo el bloque que tiene el contador de accesos más bajo. Se incrementa este contador cada vez que se accede a un bloque.

Ventajas:

- Elimina los bloques que se han accedido con menos frecuencia, lo que puede ser útil para optimizar el uso de la caché en aplicaciones con patrones de acceso irregulares.
- Puede ser muy efectivo en situaciones donde ciertos datos son consistentemente más importantes o frecuentemente utilizados que otros.
- Proporciona una buena medida de la importancia a largo plazo de los bloques de datos.

Desventajas:

- Es complicado de implementar, requiriendo hardware adicional para mantener y actualizar los contadores de frecuencia.
- Puede ser lento para adaptarse a cambios en los patrones de acceso, ya que los bloques con alta frecuencia histórica pueden permanecer en la caché incluso si ya no se utilizan.
- Puede sufrir del problema de "**envejecimiento de caché**", donde bloques nuevos pero importantes son expulsados rápidamente debido a sus bajos contadores de frecuencia.

Aleatorio:

Es un algoritmo de reemplazo en memoria caché que opera bajo el principio de seleccionar al azar un bloque para ser reemplazado cuando se necesita espacio para un nuevo bloque. Este método no considera la frecuencia de uso, el tiempo de permanencia en la caché, ni ningún otro factor para tomar la decisión de reemplazo.

Funcionamiento: Cuando se necesita espacio en la caché, simplemente selecciona de manera aleatoria uno de los bloques existentes para ser reemplazado. No requiere mantener ningún tipo de registro o contador asociado a los bloques.

Ventajas:

- Es fácil de implementar, requiriendo muy poco hardware adicional.
- Es eficiente en términos de tiempo de decisión, ya que no necesita comparar o analizar datos.
- No tiene sobrecarga (overhead) de mantenimiento de información sobre los bloques.
- Puede funcionar bien en situaciones donde los patrones de acceso son impredecibles o muy variados.

Desventajas:

- El número de bloques debe ser elevado para demostrar la eficacia, ya que con pocos bloques la probabilidad de reemplazar datos importantes aumenta.
- No garantiza un rendimiento óptimo, ya que puede reemplazar bloques frecuentemente utilizados.
- Su rendimiento puede ser inconsistente y difícil de predecir.
- No aprovecha los principios de localidad temporal o espacial que suelen ser beneficiosos en el manejo de caché.

Casos Especiales en la Gestión de Memoria Caché

Lectura de una dirección que se encuentra en la Memoria Caché (MC):

1. El controlador de caché recibe la solicitud de lectura.
2. El controlador busca la línea de caché correspondiente a la dirección solicitada usando la función de mapeo.
3. Verifica el tag de la línea para confirmar que es el dato correcto.
4. Si hay una coincidencia (HIT de caché), los datos son leídos directamente de la caché y enviados al procesador.
5. El acceso es rápido y el tiempo de respuesta es bajo.

Lectura de una dirección que NO se encuentra en la Memoria Caché (MC):

1. El controlador de caché recibe la solicitud de lectura.
2. El controlador busca la línea de caché correspondiente a la dirección solicitada usando la función de mapeo.
3. Verifica el tag de la línea y detecta que no hay coincidencia (MISS de caché).
4. La solicitud se envía a la memoria principal para leer el bloque de datos requerido.
5. El bloque de datos es cargado en la caché y el tag correspondiente es actualizado.
6. El dato es leído de la caché y enviado al procesador.
7. El acceso es más lento comparado con un HIT en caché debido a la latencia de la memoria principal.

Escritura de una dirección que se encuentra en la Memoria Caché (MC):

1. El controlador de caché recibe la solicitud de escritura.
2. El controlador busca la línea de caché correspondiente a la dirección solicitada usando la función de mapeo.
3. Verifica el tag de la línea para confirmar que es el dato correcto.
4. Si hay coincidencia (HIT de caché), los datos son escritos directamente a la línea de caché.
5. Dependiendo de la política de escritura, los cambios pueden ser inmediatamente escritos en la memoria principal (write-through) o marcados para escribir más tarde (write-back).

Políticas de Escritura:

- **Write-through:** Se escribe de manera simultánea en memoria caché y en memoria principal. Esto da la ventaja de que, al reemplazar ese bloque en memoria caché, simplemente se reemplaza rápidamente. Sin embargo, la desventaja es que cada proceso de escritura tarda más, ya que debe acceder a la memoria principal, lo que puede ralentizar el proceso si hay muchas escrituras.

- **Write-back:** Se escribe solo en memoria caché y la memoria principal se actualiza solo cuando se va a reemplazar el bloque de memoria caché. La ventaja es que la escritura es rápida, lo cual es recomendable cuando se realizan muchas actualizaciones en los datos. La desventaja es que al reemplazar un bloque de memoria caché, el proceso es más lento, porque se debe reemplazar el bloque completo en la memoria principal.

Escritura de una dirección que NO se encuentra en la Memoria Caché (MC):

1. El controlador de caché recibe la solicitud de escritura.
2. El controlador busca la línea de caché correspondiente a la dirección solicitada usando la función de mapeo.
3. Verifica el tag de la línea y detecta que no hay coincidencia (MISS de caché).
4. El dato es escrito directamente en la memoria principal sin modificar la caché, ya que no es probable que la próxima escritura sea en el bloque activo, por lo que no vale la pena traerlo a la MC.

CODIFICACION Y DECODIFICACION DE INFORMACIÓN

Códigos: Son sistemas de señales o símbolos utilizados para representar información. Cada código sigue un conjunto de reglas específicas para la conversión de datos, facilitando la comunicación y el procesamiento de información. Estos sistemas permiten transformar datos en formatos adecuados para su transmisión y recuperación, asegurando la integridad y precisión de la información en diversos contextos, desde comunicaciones digitales hasta almacenamiento de datos.

Códigos más comunes son:

- **ASCII**: Código estándar para el intercambio de información, representa caracteres como letras y números.
- **Binario**: Sistema de numeración en base 2, utilizando solo los dígitos 0 y 1.
- **BCD**: Código Decimal Codificado en Binario, representa números decimales utilizando 4 bits por dígito.
- **Gray**: Código binario en el que dos valores consecutivos difieren en un solo bit, minimizando errores en transiciones.
- **Exceso-3**: Sistema de codificación decimal en el cual cada número se representa como su valor binario más tres, útil para ciertas operaciones aritméticas.
- **7-Segmentos**: Código utilizado para la representación de números en pantallas digitales, como LCD y LED.
- **Códigos de Hamming**: Utilizan la distancia de Hamming para detectar y corregir errores, proporcionando robustez en la transmisión de datos.

Codificación:

Es el proceso de convertir datos o información en un formato específico para facilitar su almacenamiento, transmisión o compresión. Este proceso implica el uso de algoritmos que transforman el formato original de los datos en otro formato, optimizando así su manejo y protección. Los datos codificados pueden ser más eficientes para la transferencia o más seguros para el almacenamiento.

Decodificación:

Es el proceso de convertir datos codificados de vuelta a su formato original o a un formato comprensible. Este proceso utiliza algoritmos o métodos específicos para revertir la transformación realizada durante la codificación, permitiendo así que los datos sean interpretados o utilizados de manera adecuada. La decodificación asegura que la información transmitida o almacenada pueda ser recuperada correctamente.

Códigos detectores:

Los códigos detectores de errores son mecanismos que permiten identificar la presencia de errores en la información recibida. Funcionan añadiendo bits redundantes a los datos transmitidos, conocidos como bits de verificación o bits de paridad. Estos bits adicionales se calculan a partir de los datos originales y se utilizan para comprobar la integridad de la información en el receptor.

Cuando el receptor recibe la información, realiza una verificación utilizando los bits redundantes. Si los datos no coinciden con lo esperado según los bits de verificación, se detecta que ha ocurrido un error. Sin embargo, los códigos detectores solo pueden informar de la existencia de un error, sin proporcionar información sobre su ubicación específica. En este caso, el receptor debe solicitar una retransmisión de los datos para obtener la información correcta. Un ejemplo común de un código detector de errores es la suma de verificación o checksum.

Códigos correctores:

Los códigos correctores de errores van un paso más allá al no solo detectar la presencia de errores en la información recibida, sino también al corregirlos sin necesidad de solicitar una retransmisión. Para lograr esto, se añaden suficientes bits redundantes a los datos transmitidos, permitiendo al receptor no solo detectar la existencia de errores, sino también determinar su ubicación exacta y corregirlos.
Los códigos correctores de errores se diseñan de manera que los bits de verificación permiten identificar y corregir un número limitado de errores. Por ejemplo, un código Hamming puede corregir errores de un solo bit y detectar errores de dos bits. Estos códigos son esenciales en sistemas donde la retransmisión de datos no es viable o es costosa, como en comunicaciones satelitales o almacenamiento en memorias de computadora.

Código Hamming:

El código Hamming es un tipo de código corrector de errores que se utiliza para detectar y corregir errores de un solo bit en datos transmitidos o almacenados. Este se basa en la inserción de bits de paridad en posiciones específicas dentro de la palabra de datos, de modo que cualquier cambio en un solo bit de la palabra de datos puede ser identificado y corregido.
Para construir un código Hamming, se determinan las posiciones de los bits de paridad, que son potencias de dos (por ejemplo, 1, 2, 4, 8, etc.). Los bits de paridad se calculan de manera que cubren ciertas combinaciones de bits en la palabra de datos. Cuando se recibe la palabra de datos, se verifica cada bit de paridad. Si alguno de los bits de paridad no coincide, se indica la posición del bit erróneo, permitiendo al receptor corregir el error. La distancia mínima de Hamming para un código Hamming es tres, lo que significa que puede detectar hasta dos errores de un solo bit y corregir uno.

Temas generales:

1. **Entrada/Salida [Completo]- Tanenbaum– Capítulo 2 y 5 (Hoja 89 y 356)- (Página 56 y 190 en pdf)**
2. **Interrupciones [Completo]- Tanenbaum– Capítulo 5 (Hoja 379)- (Página 201 en pdf)**
3. **Secuenciadores [Completo]- W. Stallings– Capítulo 16 (Hoja 618)- (Página 640 en pdf)**
4. **Paralelismo [Completo]- Tanenbaum– Capítulo 8 (Hoja 524)- (Página 274 en pdf)**
5. **Arquitectura de microcontroladores [Completo]- W. Stallings– Capítulo 13 (Hoja 485)- (Página 507 en pdf)- De internet y de apuntes viejos**
6. **Pipeline [Completo]- W. Stallings– Capítulo 12 (Hoja 449)- (Página 471 en pdf)**
7. **Memoria Caché [Completo]- Tanenbaum– Capítulo 2 y 4 (Hoja 65)- (Página 44 en pdf)**

COMPLEMENTO: TEMAS QUE FALTAN EN EL RESUMEN

Este complemento se agrega a continuación de tu resumen. Cubre los temas de la lista del profe que no estaban o estaban incompletos, agrega el **porqué de cada tema** (que el profe pide expresamente) y corrige algunos detalles. Se basa en el contenido estándar de Stallings y Tanenbaum; cotejalo con los capítulos que tenés marcados en la última página de tu resumen.

Qué se agrega

- **A.** El porqué de cada tema (visión global).
- **B.** Arquitecturas superescalares y supersegmentadas (tema que no estaba).
- **C.** Interrupciones: ciclo de interrupción, hardware vs software, retorno con RETI, ejemplo de anidamiento.
- **D.** Pipeline: fórmula de aceleración, los tres riesgos, predicción de saltos con 2 bits.
- **E.** RISC: por qué surgió y ventanas de registros.
- **F.** Memoria caché: formato de dirección con ejemplo, niveles, tipos de fallos, tiempo medio de acceso, write-allocate.
- **G.** Entrada/Salida: modos de DMA y canales de E/S.
- **H.** Secuenciadores: formato de la microinstrucción y ciclo de ejecución.
- **I.** Paralelismo: ley de Amdahl, multithreading, multinúcleo, SIMD vectorial, clusters.
- **J.** Microcontroladores: componentes, periféricos y por qué existen.
- **K.** Códigos: ejemplo resuelto de Hamming.
- **L.** Correcciones a tu resumen.
- **M.** Ejercicios resueltos y preguntas orales.

A. El porqué de cada tema

El profe espera, además de describir cómo se compone y cómo funciona cada tema, una **visión global**: por qué existe y qué objetivo persigue.

Tema	Por qué existe / objetivo	Idea que lo justifica
Interrupciones	Que la CPU no pierda tiempo esperando a dispositivos de E/S mucho más lentos (evitar la espera activa)	Se suspende el programa, se atiende el evento y se vuelve exactamente donde se estaba (transparencia)
Secuenciador cableado	Máxima velocidad de la unidad de control	Señales generadas directamente por lógica dedicada; ideal con repertorio chico (RISC)
Secuenciador microprogramado	Controlar la complejidad de repertorios grandes con flexibilidad y diseño ordenado	Cada instrucción es un microprograma en una memoria de control; se modifica sin tocar el hardware (CISC)
Pipeline	Mejorar el rendimiento (throughput)	Solapar las etapas de varias instrucciones; ideal: una instrucción terminada por ciclo
Superescalar	Superar el límite de una instrucción por ciclo	Varios pipelines y unidades en paralelo para explotar el paralelismo entre instrucciones (ILP)
Supersegmentada	Aumentar la frecuencia de reloj	Más etapas, cada una más corta
Microcontrolador	Sistemas embebidos de bajo costo, bajo consumo y poco espacio	CPU, memoria y periféricos integrados en un chip
E/S: interrupciones y DMA	Liberar progresivamente a la CPU	Polling, luego interrupciones, luego DMA, luego canales
RISC	Reloj rápido y pipeline eficiente, aprovechando lo que los programas realmente usan	Estadísticas: predominan instrucciones simples, variables locales y llamadas a procedimientos
Memoria caché	Tapar la diferencia de velocidad entre CPU y memoria principal	**Localidad de referencia**: sin ella la caché no mejoraría el rendimiento
Paralelismo	Seguir mejorando el rendimiento cuando se agotaron la frecuencia y el ILP	Varios procesadores o núcleos trabajando a la vez

B. Arquitecturas superescalares y supersegmentadas

Esquema para contestar
Qué es: dos formas de superar el límite de una instrucción por ciclo del pipeline simple.
Cómo funciona: la superescalar replica pipelines y unidades de ejecución para emitir varias instrucciones por ciclo; la supersegmentada subdivide las etapas para acelerar el reloj.
Por qué existe: para aumentar el rendimiento aprovechando el paralelismo a nivel de instrucción (ILP).

Superescalar

El procesador tiene **múltiples unidades de ejecución y múltiples pipelines en paralelo**, lo que le permite **emitir y ejecutar más de una instrucción por ciclo**. Explota el **paralelismo a nivel de instrucción (ILP)**: instrucciones independientes de un mismo programa que pueden ejecutarse a la vez.

Etapas típicas: búsqueda de varias instrucciones por ciclo, decodificación, **emisión (issue)** a las unidades funcionales libres, ejecución y escritura/retiro (commit).

Limitaciones al paralelismo de instrucciones

Limitación	Qué es	Cómo se trata
Dependencia verdadera de datos (RAW)	Una instrucción necesita un resultado que otra aún no produjo	No se elimina: esperar o adelantar (forwarding)
Dependencia procedimental	Los saltos: no se sabe qué instrucciones vienen después	Predicción de saltos, ejecución especulativa
Conflicto de recursos	Dos instrucciones necesitan la misma unidad funcional	Duplicar unidades
Dependencia de salida (WAW)	Dos instrucciones escriben el mismo registro	Renombrado de registros
Antidependencia (WAR)	Una instrucción escribe un registro que una anterior aún no leyó	Renombrado de registros

WAW y WAR son **dependencias de nombre**: no circula un dato real, solo se reutiliza el nombre de un registro, por eso se pueden eliminar.

Políticas de emisión (de menos a más agresiva)

- **Emisión en orden, finalización en orden:** simple, pero deja unidades ociosas.
- **Emisión en orden, finalización desordenada:** las instrucciones rápidas pueden terminar antes. Hay que cuidar las dependencias de salida.
- **Emisión desordenada, finalización desordenada:** se usa una **ventana de instrucciones**, un buffer donde las instrucciones esperan hasta que sus operandos y la unidad funcional estén disponibles. Es la más eficiente y la más compleja.

Renombrado de registros

El hardware tiene más registros físicos que los visibles y reasigna nombres dinámicamente: dos instrucciones que usan el mismo registro lógico por casualidad (WAR, WAW) usan registros físicos distintos y se ejecutan en paralelo.

Otros conceptos

- **Ejecución especulativa:** se ejecuta más allá de un salto sin confirmar resultados hasta saber si la predicción fue correcta.
- **Retiro ordenado (commit):** aunque se ejecute fuera de orden, los resultados se confirman en el orden del programa.
- **VLIW:** alternativa en la que el compilador empaqueta instrucciones independientes en una palabra larga.

Supersegmentada

Pipeline con **muchas más etapas, cada una más corta**. Como cada etapa hace menos trabajo el reloj puede ser más rápido. Muchas operaciones necesitan menos de medio ciclo; subdividiendo las etapas se lanza una instrucción por *fracción* de ciclo.

- **Ventaja:** mayor frecuencia sin replicar hardware.
- **Costo:** más etapas implican más penalización por salto mal predicho y más riesgos de datos.
- Ejemplo clásico: MIPS R4000.

Comparación

	Superescalar	Supersegmentada
Estrategia	Replicar hardware: varias instrucciones por ciclo	Más etapas, más cortas: ciclo más corto
Dimensión que crece	Ancho	Profundidad
Qué explota	ILP entre instrucciones independientes	Mayor frecuencia de reloj
Costo principal	Hardware (unidades, ventana, renombrado)	Penalización de saltos y riesgos
Relación	Se pueden combinar: los procesadores modernos son ambas cosas	

C. Interrupciones: lo que falta

Ciclo de instrucción con interrupciones

Al ciclo básico (búsqueda → ejecución) se le agrega una **fase de interrupción**. Al terminar cada instrucción la CPU comprueba si hay una interrupción pendiente:

- Si no hay (o están inhabilitadas): sigue con la próxima instrucción.
- Si hay una pendiente y están habilitadas: **suspende el programa**, guarda el contexto, carga en el PC la dirección de la ISR y comienza a ejecutarla.

Qué se guarda y quién lo guarda

Elemento	Lo guarda	Cuándo
PC (dirección de la próxima instrucción del programa interrumpido)	Hardware, automáticamente	Al aceptar la interrupción
Palabra de estado / flags	Hardware, automáticamente	Al aceptar la interrupción
Registros de propósito general que use la ISR	Software (la ISR, con PUSH)	Al comienzo de la ISR; se restauran con POP al final

Cómo se vuelve: la instrucción de retorno

La última instrucción de la ISR es la de **retorno de interrupción (RETI / IRET)**: desapila la palabra de estado y el PC, y la CPU continúa exactamente en la instrucción siguiente a la última que ejecutó el programa interrumpido. La pila es **LIFO** (apuntada por el SP), por eso permite anidar: si una ISR es interrumpida, cada una apila su contexto y se desapila en orden inverso.

Hardware vs software

	Interrupción de hardware	Interrupción de software
Origen	Dispositivo externo, por una línea del bus (INTR / NMI)	Una instrucción (por ejemplo INT n) o una excepción/trampa
Sincronismo	**Asincrónica**: en cualquier momento	**Sincrónica**: en un punto previsible del programa
Uso típico	Fin de E/S, timer, falla de hardware	Llamadas al sistema, división por cero, fallo de página
Enmascarable	INTR sí (se pueden inhabilitar); **NMI** no se puede ignorar	No aplica

Clases de interrupciones (Stallings)

- **De programa:** overflow, división por cero, instrucción ilegal, acceso fuera de límites.
- **De temporizador (timer).**
- **De E/S:** fin de operación o error.
- **Por falla de hardware:** error de paridad en memoria, falla de alimentación.

Ejemplo clásico de anidamiento (Stallings)
Programa de usuario en ejecución. Prioridades: impresora = 2, comunicaciones = 4, disco = 5 (mayor número, mayor prioridad).
t=10: interrumpe la impresora y se ejecuta su ISR. t=15: llega comunicaciones (4 > 2) e interrumpe a la ISR de la impresora. t=20: llega el disco (5 > 4) e interrumpe a comunicaciones. Se atiende el disco, luego se retoma comunicaciones, luego impresora y por último el programa de usuario. La pila guarda cada contexto.

Frases clave para el oral

- Las interrupciones existen para evitar la espera activa.
- El hardware apila PC y flags; la ISR apila el resto; RETI desapila y vuelve.
- La pila LIFO permite el anidamiento.
- Secuencial = simple pero sin prioridades; anidado = respeta prioridades.

D. Pipeline: lo que falta

Rendimiento (fórmulas)

Con **k** etapas, **n** instrucciones y tiempo de etapa **t**:

- Sin pipeline: $n \cdot k \cdot t$
- Con pipeline: $[k + (n - 1)] \cdot t$
- **Aceleración = $n \cdot k / (k + n - 1)$**, que tiende a **k** cuando n es grande.

Ejemplo: k = 5, n = 100 → 500 ciclos sin pipeline y 5 + 99 = 104 con pipeline; aceleración ≈ 4,8.

> **Idea clave**
> El pipeline **no reduce el tiempo de una instrucción individual**; mejora el **throughput** (instrucciones terminadas por unidad de tiempo).

Pipeline de Stallings (6 etapas)

FI (búsqueda de instrucción), DI (decodificación), CO (cálculo de operandos), FO (búsqueda de operandos), EI (ejecución) y WO (escritura del operando). Tu resumen usa el de 5 etapas (Fetch, Decode, Execute, Memory, Write back): conviene conocer ambos.

Los tres tipos de riesgos (hazards)

Riesgo	Qué es	Soluciones
Estructural	Dos instrucciones necesitan el mismo recurso en el mismo ciclo (ej.: una sola memoria para instrucciones y datos)	Duplicar recursos; memorias o cachés separadas de instrucciones y datos (Harvard)
De datos	Una instrucción depende del resultado de otra que no terminó. **RAW** (dependencia verdadera), **WAR** y **WAW** (aparecen con ejecución fuera de orden)	Interbloqueo (stall/burbuja); **forwarding/bypassing**; reordenamiento por el compilador
De control	Saltos (20-30% de las instrucciones): las instrucciones ya cargadas pueden no servir	Predicción de saltos, salto retardado, múltiples flujos, precarga del destino, buffer de bucle

Ejemplo RAW: **ADD R1,R2,R3** seguida de **SUB R4,R1,R5**: la segunda necesita R1 antes de que la primera lo escriba. Sin forwarding hay que insertar burbujas; con forwarding el resultado pasa directo de la salida de la ALU a la entrada.

Técnicas adicionales para los saltos

- **Múltiples flujos:** duplicar las etapas iniciales y seguir ambos caminos (gasta hardware).
- **Precarga del destino del salto:** buscar también la instrucción destino para reducir la penalización.
- **Buffer de bucle:** memoria pequeña con las últimas instrucciones, ideal para bucles.

Predicción dinámica con 2 bits

Cada salto tiene asociado un contador de 2 bits: se suma si el salto se toma y se resta si no. Estados 11 y 10 predicen "se toma"; 01 y 00 predicen "no se toma". Con 2 bits hacen falta **dos fallos seguidos** para cambiar la predicción, lo que evita errores por un cambio aislado (por ejemplo, la última vuelta de un bucle).

E. Arquitecturas RISC: lo que falta

Esquema para contestar
Qué es: repertorio reducido de instrucciones simples, formato fijo, load/store, muchos registros y control cableado.
Cómo funciona: instrucciones entre registros en un ciclo; solo LOAD y STORE acceden a memoria; ventanas de registros aceleran las llamadas; el compilador optimiza.
Por qué existe: las estadísticas de programas mostraron que se usan sobre todo instrucciones simples y variables locales.

Por qué surgió RISC

Estudios sobre programas reales de alto nivel mostraron que:

- Predominan las **asignaciones** y los saltos; las operaciones complejas son poco frecuentes.
- La mayoría de los **operandos son variables locales y escalares**, que pueden guardarse en registros.
- Las **llamadas a procedimientos son muy costosas** (guardar/restaurar registros y pasar parámetros por memoria).
- Existía una **brecha semántica** entre los lenguajes de alto nivel y el repertorio de máquina; CISC intentaba cerrarla con instrucciones complejas, pero los compiladores las usaban poco.

Conclusión: conviene un repertorio pequeño, simple y rápido, con muchos registros, y dejar la complejidad en el compilador.

Ventanas de registros (Berkeley)

- El banco de registros se divide en **ventanas**; cada procedimiento usa una ventana propia.
- Cada ventana tiene registros **locales** y registros de **parámetros** que se **solapan** con las ventanas vecinas: los de salida del que llama son los de entrada del llamado.
- Pasar parámetros no requiere copiar nada ni acceder a memoria: solo cambia la ventana activa.
- Se organizan como **buffer circular**; si se agotan, se vuelcan a memoria las más viejas.
- **Alternativa:** un banco grande con registros asignados por el compilador (enfoque Stanford).

Características a recordar

- Formato de instrucción fijo y modos de direccionamiento simples.
- Arquitectura load/store.
- Casi todas las instrucciones en un ciclo.
- Muchos registros.
- Control cableado.
- Diseñada para pipeline eficiente (mejor aún con Harvard).
- Programas más largos que en CISC; la complejidad está en el compilador.

Nota sobre el pipeline RISC de tu resumen

Stallings describe primero un pipeline RISC simple de 2 etapas (I = búsqueda, E = ejecución) y después versiones de 3 y 4 etapas. En la práctica y en Tanenbaum es muy común el de 5 etapas (IF, ID, EX, MEM, WB). Decí de cuál hablás.

F. Memoria caché: lo que falta

Formato de dirección y ejemplo

Directo: **etiqueta | línea | palabra**. Totalmente asociativo: **etiqueta | palabra**. Asociativo por conjuntos: **etiqueta | conjunto | palabra**.

Ejemplo: dirección de 16 bits (memoria de 64 KB), caché de 4 KB, bloque de 16 bytes. Líneas = 4096 / 16 = 256. Bits de palabra = 4.

Mapeo	Etiqueta	Línea / Conjunto	Palabra	Cálculo
Directo	4 bits	8 bits (256 líneas)	4 bits	$16 - 8 - 4 = 4$
Totalmente asociativo	12 bits	—	4 bits	$16 - 4 = 12$
Asociativo 4 vías	6 bits	6 bits (64 conjuntos)	4 bits	$256 / 4 = 64$; $16 - 6 - 4 = 6$

A más asociatividad: etiqueta más larga y más comparadores, pero menos conflictos.

> **Ejemplo de thrashing en mapeo directo**
> Caché directa de 4 líneas; se accede alternadamente a los bloques 0 y 4. Ambos van a la línea 0 (0 mod 4 = 4 mod 4), así que cada acceso reemplaza al anterior: todos son fallos. Con asociativa por conjuntos de 2 vías (2 conjuntos), los bloques 0 y 4 van al conjunto 0 pero conviven en sus dos líneas: después de los dos primeros fallos, todo son aciertos.

Otros parámetros de diseño

- **Tamaño de línea:** más grande aprovecha la localidad espacial, pero si es excesivo hay menos líneas y se traen datos inútiles.
- **Niveles:** L1 (chica y rapidísima), L2, L3 (más grandes y lentas).
- **Unificada vs dividida:** la dividida (instrucciones y datos separados) elimina conflictos estructurales en el pipeline.
- **Tipos de fallos:** forzosos (primer acceso), de capacidad y de conflicto (típico del mapeo directo).
- **Bit de modificado (dirty bit):** en write-back marca las líneas modificadas que hay que escribir al reemplazarlas.

Tiempo medio de acceso

Con h = tasa de aciertos, Tc = tiempo de caché y Tm = tiempo de memoria principal:

- Variante simple: $\mathbf{Ta = h \cdot Tc + (1 - h) \cdot Tm}$
- Variante con acceso secuencial: $Ta = h \cdot Tc + (1 - h) \cdot (Tc + Tm)$

Ejemplo: Tc = 10 ns, Tm = 100 ns, h = 0,95 → $Ta = 0{,}95 \cdot 10 + 0{,}05 \cdot 100 = 14{,}5$ ns. Usá la variante que dieron en clase.

Escritura con fallo: dos variantes

En tu resumen aparece solo una. Hay dos: **write-no-allocate** (se escribe en memoria principal sin traer el bloque a la caché, como en tu resumen) y **write-allocate** (se trae el bloque a la caché y se escribe ahí; es lo habitual con write-back).

G. Entrada/Salida: lo que falta

Pasos de una transferencia DMA

1. La CPU programa al DMA (sentido, dirección de memoria, cantidad de palabras, dispositivo).
2. El dispositivo pide servicio al DMA.
3. El DMA toma el bus y transfiere una palabra.
4. Incrementa el registro de direcciones y decrementa el registro de cuenta.
5. Cuando la cuenta llega a cero, **interrumpe una sola vez** a la CPU.

Modos de uso del bus por el DMA

- **Robo de ciclo:** toma el bus palabra a palabra; la CPU solo espera un ciclo (no guarda contexto).
- **Ráfaga:** retiene el bus durante todo el bloque.
- **Transparente:** solo usa ciclos en que la CPU no necesita el bus.

Configuraciones de DMA (resumen)

- Bus único con DMA separado: cada palabra usa el bus dos veces.
- DMA integrado con E/S: menos ciclos de bus.
- Bus de E/S dedicado: configuración fácilmente ampliable.

Evolución posterior: canales y procesadores de E/S

Procesadores con su propio programa de E/S. La CPU solo ordena iniciar y se libera casi por completo. Tipos: selector y multiplexor.

Comparación

	Programada	Por interrupciones	DMA
La CPU durante la espera	Consulta sin parar	Hace otro trabajo	Hace otro trabajo
Quién mueve los datos	La CPU	La CPU	El controlador DMA
Interrupciones por transferencia	Ninguna	Una por palabra	Una al final del bloque
Conviene para	Sistemas simples	Pocos datos, eventos esporádicos	Grandes volúmenes (disco, red)

H. Secuenciadores: lo que falta

Formato de la microinstrucción

Una microinstrucción contiene: (1) los **bits de control** (qué microórdenes activar) y (2) la **información de la próxima microinstrucción** (dirección, o condición de salto). Se almacena en la **memoria de control**.

Componentes del secuenciador microprogramado

- Memoria de control con los microprogramas.
- Registro de dirección de la memoria de control (microPC).
- Registro de microinstrucción.
- Lógica de secuenciamiento: incrementa el microPC o salta según flags y condiciones.
- Mapeo del código de operación a la dirección inicial del microprograma.

Ciclo de ejecución

1. Se busca la instrucción de máquina y se carga en el RI.
2. El opcode se traduce a la dirección inicial de su microprograma.
3. Se ejecutan las microinstrucciones; cada una emite microórdenes y define cuál sigue.
4. Al terminar el microprograma, se vuelve a buscar la próxima instrucción.

Por qué existe cada tipo

El **cableado** existe por velocidad. El **microprogramado** existe para controlar la complejidad de repertorios grandes (CISC), con un diseño sistemático y modificable. Existe además la **nanoprogramación** (dos niveles de memoria de control) para ahorrar espacio.

I. Paralelismo: lo que falta

Ley de Amdahl

Si **f** es la fracción del programa que se puede paralelizar y **N** la cantidad de procesadores: **Speedup = 1 / [(1 − f) + f / N]**. La parte secuencial limita todo: aun con N infinito el tope es 1 / (1 − f).

Ejemplo: f = 0,9 y N = 8 → 1 / (0,1 + 0,1125) ≈ 4,7. Tope con N infinito: 10.

Multithreading y multinúcleo

- **Multithreading:** el procesador mantiene varios hilos y cambia entre ellos para ocultar latencias. Variante **SMT**: hilos simultáneos en un procesador superescalar.
- **Multinúcleo (multicore):** varios procesadores (núcleos) en un chip, con caché compartida en algún nivel.

SIMD: procesadores matriciales y vectoriales

En los **matriciales** hay muchos elementos de proceso que ejecutan la misma instrucción a la vez sobre datos distintos. Los **vectoriales** usan registros vectoriales y unidades segmentadas para operar sobre vectores. Las GPU y las extensiones SSE/AVX son SIMD.

Clusters

Conjunto de computadoras completas interconectadas que trabajan como un único recurso. Ventajas: escalabilidad, alta disponibilidad, costo bajo.

CC-NUMA

NUMA con coherencia de caché: es el NUMA que se usa en la práctica.

Por qué existe el paralelismo

Se llegó a límites físicos de frecuencia y de paralelismo a nivel de instrucción. Para seguir mejorando el rendimiento se usan varios procesadores o núcleos a la vez.

J. Microcontroladores: lo que falta

Componentes típicos

- **CPU** con su secuenciador, ALU y registros.
- **Memoria de programa** (Flash/ROM) y **de datos** (RAM); a veces EEPROM para parámetros.
- **Puertos de E/S** (GPIO), **timers/contadores**, **UART/SPI/I2C**, **conversores A/D**, **PWM**.
- **Controlador de interrupciones**: forma normal de atender eventos de periféricos.
- **Watchdog:** reinicia el sistema si el programa se cuelga.
- **Modos de bajo consumo (sleep):** el micro duerme hasta que lo despierta una interrupción.

Por qué existe

Sistemas embebidos de bajo costo, bajo consumo y poco espacio, donde una computadora completa sería excesiva.

Ejemplos

8051, PIC, AVR (Arduino), ARM Cortex-M. **Completá esta sección con el modelo que vieron en clase** (registros, mapa de memoria, repertorio, periféricos).

K. Códigos: ejemplo resuelto de Hamming

Hamming (7,4) con paridad par. Posiciones 1 a 7: p1 p2 d1 p3 d2 d3 d4. Cada bit de paridad en la posición 2^k cubre las posiciones cuyo número binario tiene el bit k en 1: p1 cubre 1,3,5,7; p2 cubre 2,3,6,7; p3 cubre 4,5,6,7.

- Datos: 1011 (d1=1, d2=0, d3=1, d4=1).
- p1: bits 3,5,7 = 1,0,1 → suma 2 → p1 = 0.
- p2: bits 3,6,7 = 1,1,1 → suma 3 → p2 = 1.
- p3: bits 5,6,7 = 0,1,1 → suma 2 → p3 = 0.
- Palabra transmitida: **0110011**.

Supongamos que llega con error en la posición 5: **0110111**. Se recalculan los chequeos:

- Grupo de p1 (1,3,5,7) = 0,1,1,1 → impar → 1.
- Grupo de p2 (2,3,6,7) = 1,1,1,1 → par → 0.
- Grupo de p3 (4,5,6,7) = 0,1,1,1 → impar → 1.

Síndrome (p3 p2 p1) = 101 = 5 → el error está en la posición 5. Se invierte ese bit y se corrige.

L. Correcciones y matices a tu resumen

Dónde	Dice	Corrección / matiz
Coherencia: MESI	MOESI → O = Ocupado	La O es **Owned** (propietario): la línea está modificada y compartida, y esta caché es responsable de proveer el dato y escribirlo en memoria.
Secuenciadores	El microprogramado "desapareció con la arquitectura RISC"	Es más defendible decir que **perdió protagonismo** en RISC. Hoy los x86 usan microcódigo para instrucciones complejas y traducen internamente a microoperaciones tipo RISC.
Códigos: Hamming	Puede detectar hasta dos errores y corregir uno	Con distancia mínima 3 se **corrige 1 error o se detectan 2**, no ambas cosas a la vez. Para corregir 1 y detectar 2 (SEC-DED) se agrega un bit de paridad global y la distancia pasa a 4.
Caché: escritura con fallo	Se escribe directo en memoria sin traer el bloque	Es la variante **write-no-allocate**. También existe **write-allocate**.
Pipeline en RISC	Solo 2 etapas principales	Es la versión simple de Stallings. También se ven 3, 4 y 5 etapas (IF, ID, EX, MEM, WB).
Pipeline CISC	5 etapas	Stallings usa 6 (FI, DI, CO, FO, EI, WO). Conocé ambas.
Pipeline	"Reduce el tiempo de ciclo por instrucción"	El pipeline mejora el **throughput**, no la latencia de una instrucción individual.
Interrupciones secuenciales	Se atienden una a la vez	En Stallings se llama **inhabilitar interrupciones** durante la atención (enfoque secuencial).

M. Ejercicios resueltos y preguntas orales

Ejercicio 1: campos de dirección en caché

Dirección de 20 bits, caché de 8 KB, bloque de 32 bytes. Palabra: 5 bits. Líneas: 8192 / 32 = 256.

- **Directo:** línea 8 bits; etiqueta = 20 − 8 − 5 = **7 bits**.
- **Asociativo:** etiqueta = 20 − 5 = **15 bits**.
- **2 vías:** conjuntos = 128 → 7 bits; etiqueta = 20 − 7 − 5 = **8 bits**.

Ejercicio 2: aceleración de un pipeline

6 etapas, 50 instrucciones sin riesgos. Sin pipeline: 300 ciclos. Con pipeline: 6 + 49 = 55 ciclos. Aceleración = 300 / 55 ≈ **5,45**.

Ejercicio 3: ley de Amdahl

Programa con 20% secuencial (f = 0,8). Con N = 4: 1 / (0,2 + 0,2) = **2,5**. Con N infinito: 1 / 0,2 = **5**.

Ejercicio 4: tiempo medio de acceso

Tc = 10 ns, Tm = 100 ns, h = 0,95 → 0,95 · 10 + 0,05 · 100 = **14,5 ns**.

Ejercicio 5: riesgo RAW en pipeline de 5 etapas

- Sin forwarding: la SUB necesita R1 en decodificación y la ADD lo escribe en WB; hacen falta **2 burbujas** (suponiendo escritura en la primera mitad del ciclo y lectura en la segunda).
- Con forwarding: **0 burbujas** para ALU a ALU.
- Caso load-use (carga seguida de uso): aun con forwarding queda **1 burbuja**.

Ejercicio 6: DMA vs interrupciones

Transferir 1000 palabras. Por interrupciones: 1000 interrupciones, cada una con guardado y restauración de contexto. Con DMA: 1 sola interrupción al final.

Preguntas orales

1. Describí paso a paso qué pasa desde que llega una interrupción hasta que se retoma el programa. ¿Qué rol cumple la pila? ¿Qué hace RETI?
2. Interrupciones anidadas vs secuenciales. ¿Quiénes pueden interrumpir? Hardware vs software.
3. Secuenciador cableado vs microprogramado. ¿Por qué cada uno se asocia a RISC o CISC?
4. ¿Qué es el pipeline y por qué existe? ¿Por qué no reduce la latencia de una instrucción?
5. Riesgos del pipeline, ejemplos y soluciones. Predicción de saltos y salto retardado.
6. Superescalar vs supersegmentada. ¿Qué limita el ILP? ¿Qué es el renombrado de registros?
7. Microcontrolador vs microprocesador. Harvard vs von Neumann.
8. E/S programada, por interrupciones y DMA. ¿Qué es el robo de ciclo?
9. ¿Por qué existió RISC? Ventanas de registros.

10. ¿Por qué funciona la caché? Mapeo, reemplazo y escritura.

11. Flynn. Multiprocesadores vs multicomputadoras. SMP vs NUMA.

12. Coherencia de caché: soluciones y MESI.

13. Ley de Amdahl.